Design and Verification of a Lightweight RISC-V-Based IoT Security Processor with Iterative Hardware-Reusable AES 128

Dissertation

submitted in partial fulfillment of the requirements for the degree of

Master of Technology

by

Varada Inamdar

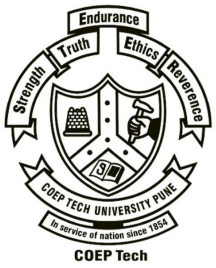
(MIS No. 712438020)

Guide

Dr. Ashwini Kulkarni

Verification Guidance

Ms. Ashlesha Gokhale



VLSI Design

Department of Electronics and Telecommunication Engineering

COEP Technological University, Pune

Department of Electronics and Telecommunication Engineering
COEP Technological University, Pune

CERTIFICATE

This is to certify that the thesis titled "**Design and Verification of a Lightweight RISC-V-Based IoT Security Processor with Iterative Hardware-Reusable AES 128**" submitted by **Varada Inamdar** (712438020) in partial fulfillment of the requirements for the award of Master of Technology degree in Electronics Engineering, with specialization in VLSI Design during year 2024–26 at COEP Technological University, Pune is an authentic work carried out under my supervision and guidance.

Dr. Ashwini Kulkarni Project Guide	**Dr. Vibha Vyas** Head of Department of Electronics and Telecommunication Engineering
Date:	**Place:**

Dissertation Approval

This dissertation entitled

Design and Verification of a Lightweight RISC-V-Based IoT Security Processor with Iterative Hardware-Reusable AES 128

by

Varada Inamdar

is approved for the degree of

Master of Technology with Specialization in VLSI Design

of

Department of Electronics and Telecommunication Engineering
COEP Technological University, Pune

Examiners	**Name**	**Signature**
1. External Examiner		
2. Guide/Supervisor	Dr. Ashwini Kulkarni	

ACKNOWLEDGEMENT

I express my sincere gratitude to my guide, Dr. Ashwini Kulkarni, for continuous guidance, constructive criticism, and support throughout this work. I thank Dr. Vaishali Ingle, Faculty Coordinator, for her academic guidance and coordination during the M.Tech project. I also thank Dr. Vibha Vyas, Head of the Department of Electronics and Telecommunication Engineering, and the faculty and laboratory staff of COEP Technological University for providing access to FPGA, Questa, Quartus, and Cadence design tools. I am thankful to Ms. Ashlesha Gokhale for guidance in verification and UVM methodology, and to my classmates, friends, and family for their encouragement during the development and verification of this project [1, 2].

ABSTRACT

This thesis presents the design and verification of a lightweight RISC-V based IoT security processor using an iterative hardware-reusable AES-128 accelerator. The work builds on the AES research titled *Iterative and Hardware-Reusable AES-128 Architecture for FPGA and IoT Applications* and extends it into a processor-level security system with AES-CTR, MMIO peripherals, a custom security ISA, UART output, DMA-lite, interrupt handling, and activity counters. The final application context is an autonomous vehicle sensor-security node, where safety-relevant telemetry such as IMU, wheel-speed, radar metadata, or vehicle-health samples must be protected before it leaves the local electronic control unit [3, 4].

The processor is implemented as a five-stage RV32I-style pipeline integrated with a reusable AES core. The AES engine is used in ECB mode for known-answer compatibility and in CTR mode for stream-friendly sensor encryption. The work contains three intentionally separate result sets. First, the baseline and proposed standalone AES architectures are compared using Cadence Genus with a 55 nm standard-cell library. Second, the complete AES plus RISC-V processor is functionally verified in Questa and implemented in Quartus for FPGA resource and timing analysis. Third, the complete SoC is synthesized in Cadence Genus using GPDK045 high-threshold-voltage cells to study low-leakage ASIC-style mapping, timing closure, cell area, and vectorless power. Directed tests, randomized smoke tests, UVM end-to-end testing with an independent C-DPI AES-CTR reference model, portable functional coverage, and a full SoC scenario exercise the processor and its sensor, DMA, AES, UART, interrupt, power, and sleep behavior [5, 6].

Keywords: RISC-V, AES-128, AES-CTR, IoT security, autonomous vehicle telemetry, FPGA, UVM, MMIO, custom ISA, low-power VLSI.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS AND SYMBOLS

Term	Meaning
AES	Advanced Encryption Standard
CTR	Counter mode of operation
ECB	Electronic Codebook mode
RTL	Register-transfer level
MMIO	Memory-mapped input/output
DMA	Direct memory access
UVM	Universal Verification Methodology
DPI	Direct Programming Interface between SystemVerilog and C
ALM	Adaptive Logic Module
FPGA	Field-programmable gate array
ASIC	Application-specific integrated circuit
FSM	Finite-state machine
UART	Universal asynchronous receiver/transmitter
SPI	Serial Peripheral Interface
IRQ	Interrupt request
SDC	Synopsys Design Constraints
SDF	Standard Delay Format
P_{dyn}	Dynamic power dissipated by switching activity
P_{static}	Static or leakage-related power dissipation
T_{AES}	AES data throughput
N_c	Number of clock cycles required for one AES block
f_{clk}	Applied clock frequency
D_{crit}	Critical combinational path delay

CHAPTER 1:

INTRODUCTION

The growth of autonomous and connected platforms has made small embedded processors responsible for sensor movement, local decisions, and communication. In an autonomous vehicle, a low-level sensor node may collect IMU samples, wheel-speed information, steering data, battery status, or radar object metadata before sending it to a gateway ECU. During project bring-up, even a simple sensor-to-UART path made one practical point very clear: raw data is easy to observe at internal buses and serial outputs unless security is placed close to the source. This observation motivates a compact processor where encryption is not an external afterthought but part of the local data path [2, 4].

AES remains one of the most widely used symmetric ciphers for embedded security because it has a fixed block size, a well-understood mathematical structure, and long industrial adoption. However, the implementation style matters. Software AES is flexible but consumes processor cycles, while fully parallel hardware AES improves speed at the cost of area and switching activity. Compact and lightweight implementations therefore remain important for small edge nodes that must protect data without becoming too large, hot, or power hungry [9, 23, 26, 30].

The first part of this work studies an iterative and hardware-reusable AES-128 architecture. Instead of instantiating ten round blocks or a deep fully unrolled pipeline, the proposed AES design reuses functional hardware across rounds under finite-state-machine control. This follows the broader compact-hardware principle of reducing duplicated substitution and round logic, while accepting additional latency for lower implementation cost [25, 27, 29]. The RTL is written in SystemVerilog so that the same architectural intent can be simulated, verified, and synthesized through contemporary digital-design flows [7].

The second part integrates this AES engine with a five-stage RISC-V processor. The processor accesses AES, sensor/SPI, UART, interrupt, DMA-lite, and power-management blocks through memory-mapped registers. A custom security ISA path is also added so selected AES operations can be triggered as processor instructions. This combination allows the thesis to compare conventional MMIO accelerator control with instruction-level security acceleration in a single design [8, 12].

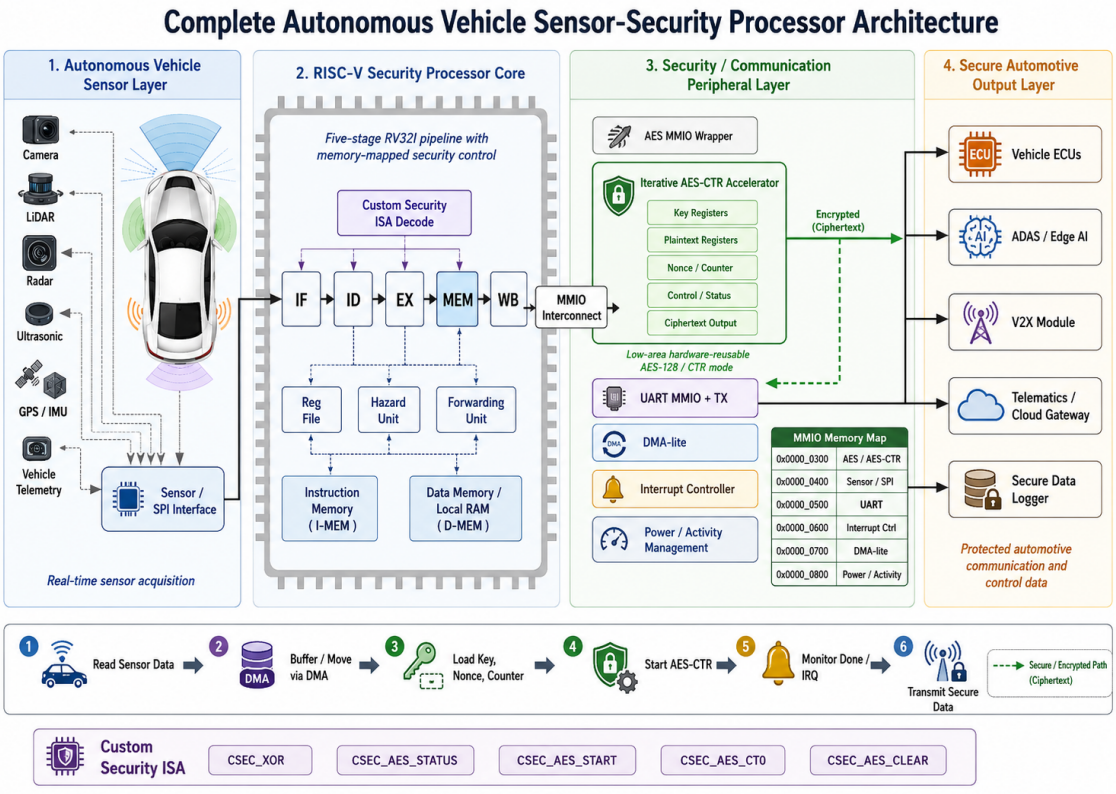


Figure 1.1: Complete autonomous vehicle sensor-security processor architecture showing the sensor layer, five-stage RISC-V processor, MMIO security peripherals, iterative AES-CTR accelerator, secure outputs, and custom security ISA.

1.1 Objectives

The main objective is to show that an iterative AES engine can be reused not only as a standalone low-power cryptographic block but also as the security accelerator inside a RISC-V based embedded processor. The design must preserve the original AES known-answer behavior, support AES-CTR for streaming sensor data, expose measurable low-power activity counters, and remain synthesizable for FPGA and ASIC-style synthesis flows [3, 5].

A second objective is verification completeness. The processor is not considered complete only because AES produces one correct ciphertext. The CPU pipeline, MMIO decoding, custom instructions, UART serial output, DMA movement, interrupt status, power counters, randomized data, and UVM reference comparison must all be checked. This is why the report treats simulation transcripts and waveform screenshots as first-class verification evidence [1, 2].

CHAPTER 2:

AES-128 BACKGROUND AND DESIGN CHOICES

2.1 Security and Cryptographic Foundations

The processor begins from a practical security requirement: sensor information must remain confidential while it moves from the local acquisition interface to an external receiver. Figure 2.1 places this requirement within the wider cryptographic context and also makes clear that encryption is one security mechanism rather than a complete security policy [19, 30].

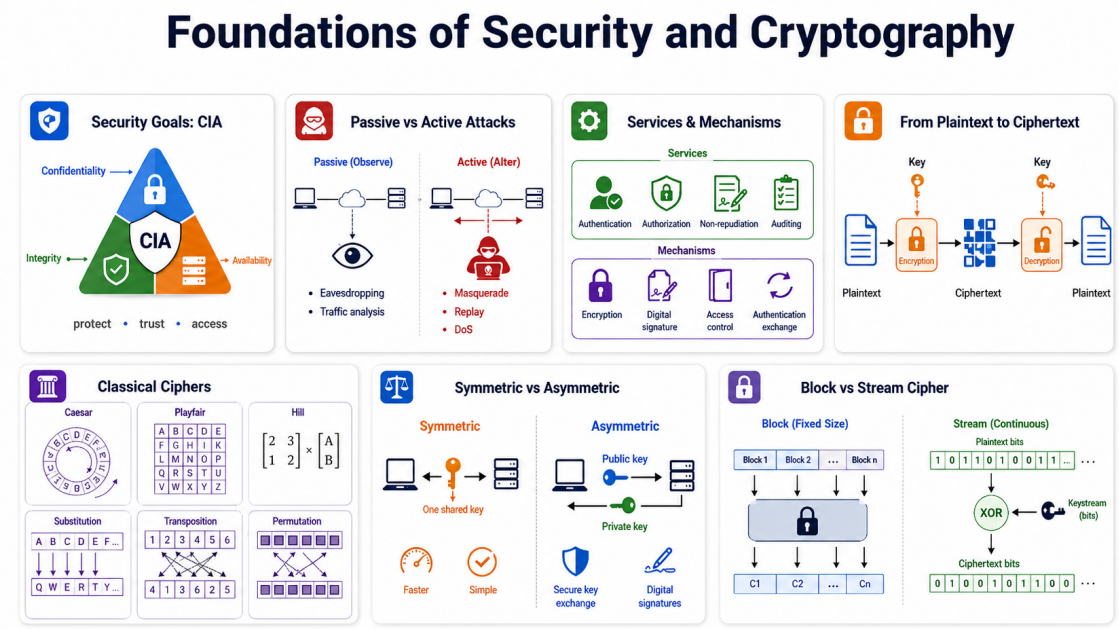


Figure 2.1: Foundations of security and cryptography used to position the proposed processor. The CIA model separates confidentiality, integrity, and availability; passive attacks observe communication, whereas active attacks alter, replay, impersonate, or disrupt it. Security services such as authentication, authorization, non-repudiation, and auditing are realized through mechanisms including encryption, digital signatures, access control, and authenticated exchanges. The plaintext-to-ciphertext path illustrates reversible key-controlled protection. Classical substitution and transposition concepts lead to modern symmetric and asymmetric schemes. This work selects symmetric AES-128 because a shared-key hardware engine is compact and fast for embedded telemetry, and uses CTR operation to obtain stream-like XOR processing from a standardized block cipher. AES-CTR provides confidentiality; message authentication, protected key storage, and replay prevention remain necessary production extensions.

2.2 AES-128 Transformations

AES is a substitution-permutation network operating on a 128-bit state arranged as sixteen bytes. For AES-128, the cipher uses a 128-bit key and ten rounds. Each normal round applies SubBytes, ShiftRows, MixColumns, and AddRoundKey. The final round omits MixColumns. The security comes from nonlinear byte substitution, byte permutation, finite-field mixing, and repeated key addition [24, 30].

The AES state can be written as a four by four byte matrix. If P is plaintext and K_0 is the first round key, the initial state is

$$S_0 = P \oplus K_0. \tag{2.1}$$

For rounds 1 to 9, the state is transformed as

$$S_r = \text{AddRoundKey}(\text{MixColumns}(\text{ShiftRows}(\text{SubBytes}(S_{r-1}))), K_r). \tag{2.2}$$

In the final round,

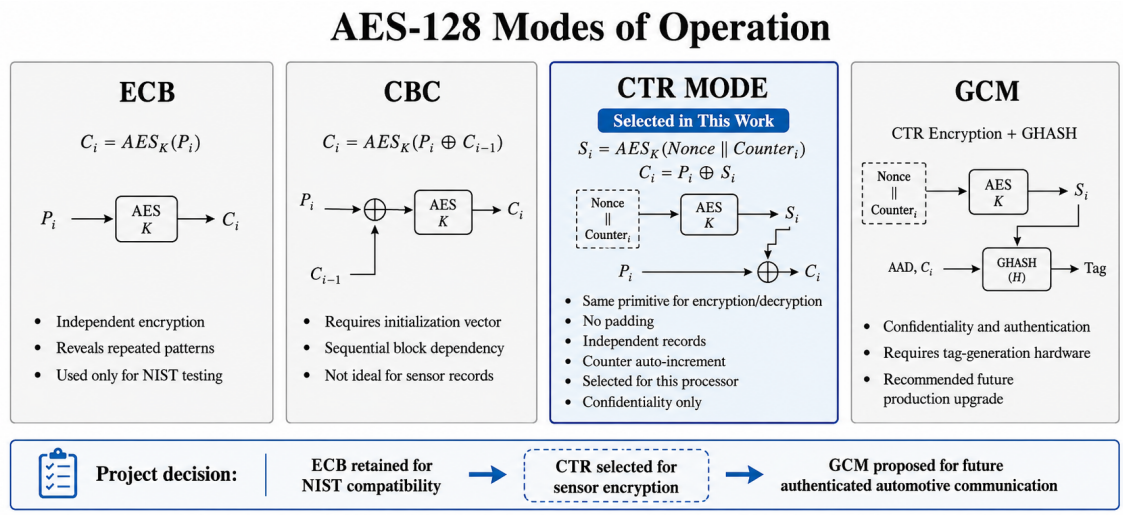
$$C = \text{AddRoundKey}(\text{ShiftRows}(\text{SubBytes}(S_9)), K_{10}). \tag{2.3}$$

These equations are simple, but the hardware cost depends strongly on how many S-boxes, round datapaths, and key-expansion blocks are active at one time [26, 28].

Listing 2.1: AES-128 encryption using an iterative reusable datapath

```
Load plaintext P and key K
S = P XOR K0
for round = 1 to 9:
    S = SubBytes(S)
    S = ShiftRows(S)
    S = MixColumns(S)
    Generate next round key Kr
    S = S XOR Kr
Final round:
    S = SubBytes(S)
    S = ShiftRows(S)
    Generate K10
    C = S XOR K10
Assert done and expose ciphertext C
```

AES can be used in several modes. ECB is useful for known-answer testing because a single plaintext and key always produce a deterministic ciphertext, but it is not suitable for repeated structured sensor records because equal plaintext blocks produce equal ciphertext blocks. CTR mode turns AES into a keystream generator by encrypting a nonce-counter value and XORing the result with plaintext. This is attractive for sensor telemetry because encryption and decryption use the same AES encrypt primitive, no padding is required for a stream of records, and the counter can be advanced block by block [25, 30].



Comparison of AES operating modes and selection of CTR mode for lightweight sensor-data encryption. ECB is retained for known-answer verification, while GCM is identified as a future authenticated-encryption extension.

Figure 2.2: AES mode-selection summary. ECB encrypts blocks independently and is retained only for deterministic NIST known-answer testing. CBC introduces block chaining and is less convenient for independent telemetry records. CTR generates a keystream from the nonce and counter, requires no padding, reuses the AES encrypt primitive for encryption and decryption, and is therefore selected for this processor. GCM extends CTR with authentication and is the preferred future production upgrade.

The AES-CTR operation used in the processor is

$$KS_i = E_K(N \parallel CTR_i), \quad C_i = P_i \oplus KS_i, \tag{2.4}$$

where N is the nonce and CTR_i is the counter for block i. Decryption is identical in structure:

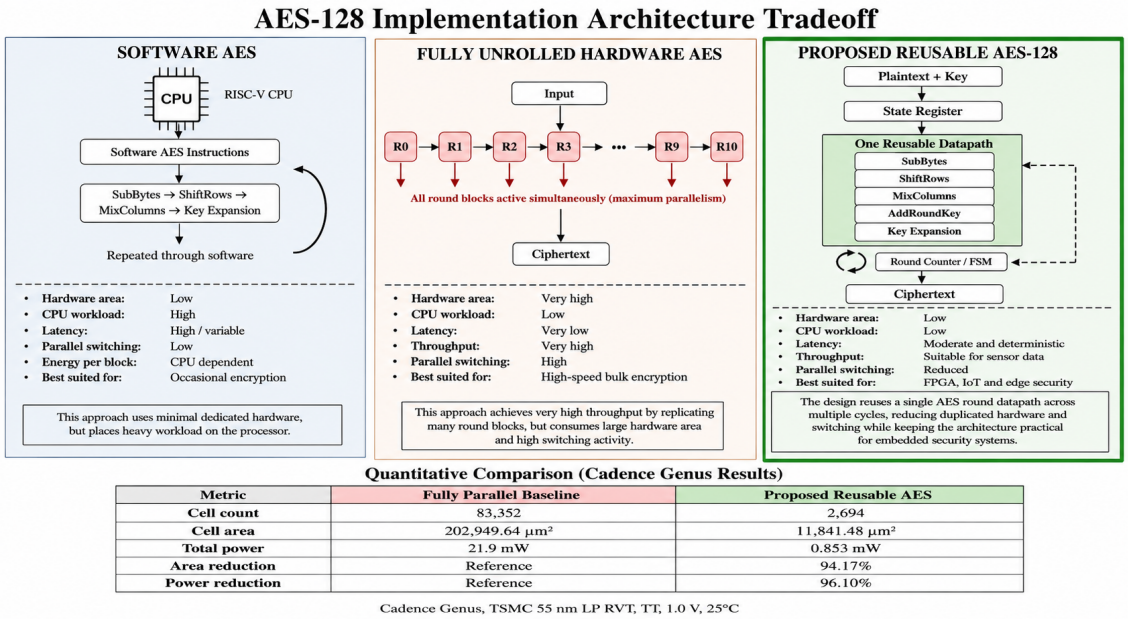
$$P_i = C_i \oplus E_K(N \parallel CTR_i). \tag{2.5}$$

This symmetry makes the hardware wrapper simpler because only the AES encrypt datapath is required for both directions [22, 30].

Listing 2.2: AES-CTR sensor-record encryption

```
Read sensor record Pi from sensor/SPI MMIO or RAM
Load AES key K, nonce N, and counter CTRi
KS_i = AES_Encrypt_K(N || CTRi)
C_i  = Pi XOR KS_i
Transmit or store C_i
CTR_(i+1) = CTR_i + 1
```

Traditional fully unrolled AES minimizes latency by duplicating round hardware, while software AES minimizes dedicated hardware but keeps the CPU active for many instructions. The proposed accelerator occupies the practical middle ground for moderate-rate sensor security by reusing one deterministic round datapath [26, 27].



Quantitative Comparison (Cadence Genus Results)

Metric	Fully Parallel Baseline	Proposed Reusable AES
Cell count	83,352	2,694
Cell area	202,949.64 µm²	11,841.48 µm²
Total power	21.9 mW	0.853 mW
Area reduction	Reference	94.17%
Power reduction	Reference	96.10%

Cadence Genus, TSMC 55 nm LP RVT, TT, 1.0 V, 25°C

Figure 2.3: Architectural tradeoff among three AES implementation styles. Software AES requires little dedicated logic but consumes processor cycles and has workload-dependent latency. A fully unrolled implementation duplicates the ten round stages for high throughput and low latency at the cost of maximum area and concurrent switching. The proposed accelerator keeps CPU workload low while reusing SubBytes, ShiftRows, MixColumns, AddRoundKey, and key-expansion hardware across rounds. The embedded Cadence Genus comparison reports 2,694 cells, 11,841.48 μm^2, and 0.853 mW for the reusable core, corresponding to approximately 94.17% lower area and 96.10% lower total power than the parallel baseline.

CHAPTER 3:

RESEARCH GAP AND PROPOSED DIRECTION

Recent AES hardware studies show a continued need for efficient FPGA and embedded cryptographic accelerators. Some designs focus on configurability, dynamic reconfiguration, side-channel or power-trace analysis, and some on high throughput [13, 16]. The research gap addressed here is narrower and practical: how to reuse AES hardware to reduce area and switching, then integrate that low-power primitive into a processor where sensor data can be encrypted before transmission [1–3].

Dynamic power in CMOS logic is commonly approximated by

$$P_{dyn} = \alpha C_L V^2 f, \tag{3.1}$$

where α is switching activity, C_L is load capacitance, V is supply voltage, and f is clock frequency. A fully parallel AES architecture can reduce time but may increase α and effective switched capacitance by activating more logic. The iterative AES design reduces the number of active round blocks per cycle and therefore targets activity and capacitance at the architectural level [18, 19].

For a 128-bit AES block completed in N_c cycles at clock frequency f_{clk}, block throughput is

$$T_{AES} = \frac{128 f_{clk}}{N_c}. \tag{3.2}$$

The current reusable AES implementation was measured from the UVM waveform between `aes\_start\_pulse` and the primitive `aes\_done` edge. The observed primitive latency is 295 clock periods; the MMIO wrapper asserts its software-visible `done\_reg` one period later, after 296 clock periods. At the FPGA target of 50 MHz, the primitive block throughput is

$$T_{AES,\text{primitive}} = \frac{128 \times 50 \times 10^6}{295} \approx 21.69 \text{ Mbit/s}. \tag{3.3}$$

The corresponding primitive latency is

$$L_{AES,\text{primitive}} = \frac{295}{50 \times 10^6} = 5.90\ \mu s. \tag{3.4}$$

For software polling through MMIO, the measured 296-cycle completion latency corresponds to 5.92 μs and approximately 21.62 Mbit/s. These are cycle-accurate RTL measurements projected to the 50 MHz FPGA target, rather than measured board-level data rates. The system UART rate can still be lower than this, so the processor design treats AES as a

secure local accelerator while UART remains the practical serial-output bottleneck [22, 24].

Table 3.1: Traditional approach versus proposed processor-level approach

Aspect	Traditional method	Proposed method
Sensor data handling	CPU reads sensor and may transmit raw or software-encrypted data	CPU reads sensor and sends record through AES-CTR hardware before UART output
AES implementation	Software AES or fully parallel hardware AES	Iterative hardware-reusable AES accelerator
Control interface	Function calls or external accelerator protocol	MMIO plus custom security ISA path
Data movement	CPU copies each word	DMA-lite moves selected words
Event handling	Polling-heavy firmware	Interrupt pending, enable, and clear registers
Power observation	External estimation only	Activity counters visible through MMIO
Main limitation	Either CPU overhead or high hardware area	Higher AES latency than unrolled AES
Mitigation	Not inherent	DMA, CTR streaming, clock enables, and optional custom instruction triggers

CHAPTER 4:

PROPOSED RISC-V SECURITY PROCESSOR ARCHITECTURE

The top-level module is `riscv_aes_advancements`. It connects a five-stage RV32I-style pipeline with AES, sensor/SPI, UART, DMA-lite, interrupt, and power-management peripherals. The processor stages follow the familiar IF, ID, EX, MEM, and WB sequence, while the MEM stage acts as a small local interconnect for MMIO registers. This choice keeps the pipeline structure understandable and avoids rewriting stable CPU stages [8, 12].

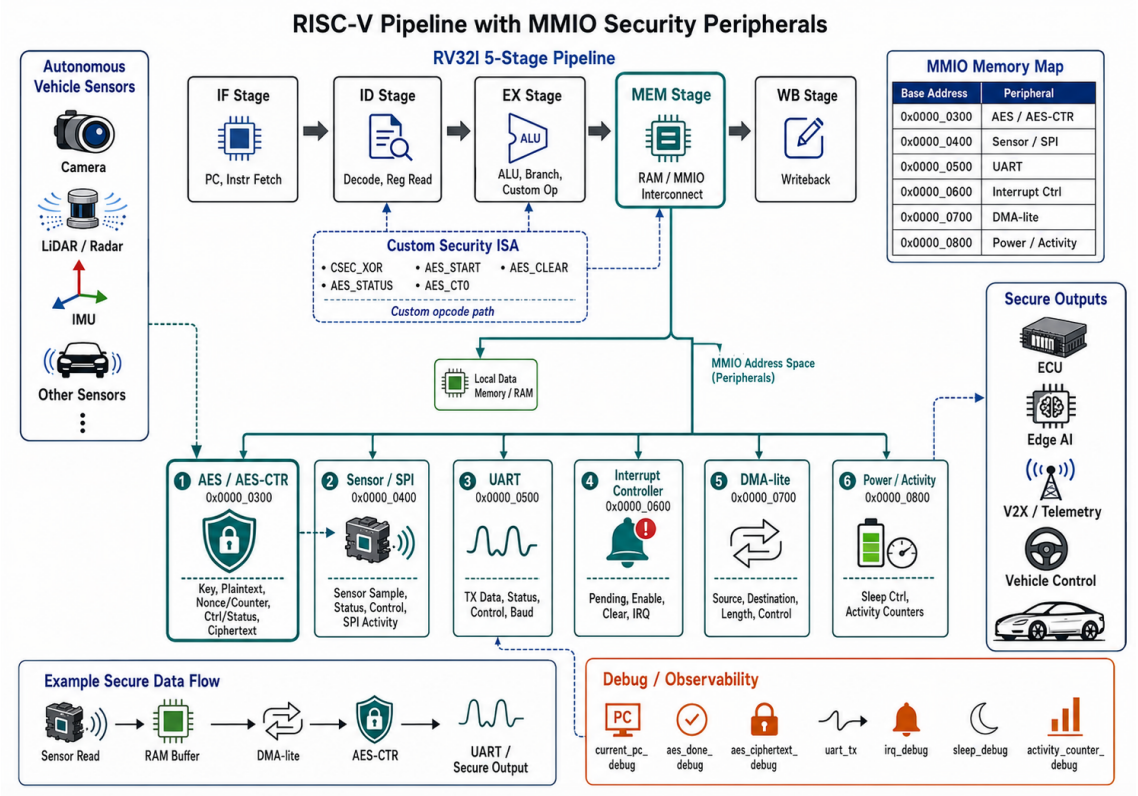


Figure 4.1: Five-stage RISC-V pipeline with MMIO security peripherals, custom instruction path, address map, secure data flow, and FPGA-observable debug outputs.

The memory map keeps AES at address range 0x00000300 for backward compatibility and adds sensor/SPI, UART, interrupt, DMA-lite, and power-management regions. In this design the CPU uses ordinary load and store instructions to control peripherals, which makes the hardware easy to test from the same directed instruction framework used for the core [4, 5].

Table 4.1: MMIO memory map of the integrated security processor

Base address	Peripheral	Main purpose
0x0000_0300	AES / AES-CTR	Key, plaintext, nonce, counter, control, status, ciphertext
0x0000_0400	Sensor / SPI	Sensor sample, status, control, SPI activity path
0x0000_0500	UART	TX data, status, control, baud divisor
0x0000_0600	Interrupt controller	Pending, enable, clear, combined IRQ
0x0000_0700	DMA-lite	Source, destination, length, control, status
0x0000_0800	Power/activity	Sleep control and activity counters

The UART output is intentionally simple. The transmitter uses an 8-N-1 serial frame: one start bit, eight data bits, and one stop bit. This framing and the programmable baud-generation principle follow conventional asynchronous UART operation [21]. In the directed Questa verification, the clock period is 10 ns and the UART divisor is programmed to `1`; therefore each serial bit occupies

$$T_{\text{bit,sim}} = (1 + 1) \times 10 \text{ ns} = 20 \text{ ns}. \tag{4.1}$$

The preserved directed-peripheral waveform records `tx\_busy` rising at 12.385 μs and falling at 12.585 μs. The observed 8-N-1 frame duration is therefore

$$T_{\text{frame,sim}} = 12.585 - 12.385 = 0.200\ \mu s, \tag{4.2}$$

which corresponds to a simulation serial rate of 50 Mbaud and an 8-bit payload rate of

$$T_{\text{UART,payload,sim}} = \frac{8}{0.200\ \mu s} = 40 \text{ Mbit/s}. \tag{4.3}$$

The sticky MMIO completion flag is observed at 12.595 μs, one 100 MHz clock after `tx\_busy` deasserts. These values describe the accelerated verification configuration and are not claimed as an automotive deployment baud rate. The busy and done behavior confirms that software can apply correct flow control [4, 22].

Listing 4.1: Autonomous vehicle sensor-to-ciphertext flow

```
CPU reads vehicle sensor sample from sensor/SPI MMIO
CPU stores sample into local RAM record buffer
DMA-lite copies selected word or record into AES input staging area
CPU writes AES key, nonce, counter, and plaintext block
CPU starts AES-CTR by MMIO control or custom security instruction
CPU polls AES done or observes interrupt pending
CPU writes ciphertext byte or record to UART TX register
Interrupt controller captures AES, UART, sensor, and DMA events
Power-management block records active cycles and sleep cycles
```

4.1 Custom Security ISA

The custom ISA extension uses the RISC-V custom opcode space to create compact security operations without disturbing standard RV32I instructions. The design includes custom operations such as XOR, AES status read, AES start, ciphertext read, and AES clear. These instructions provide a direct path for security control while keeping the existing MMIO interface available for software compatibility and detailed debug [8, 12].

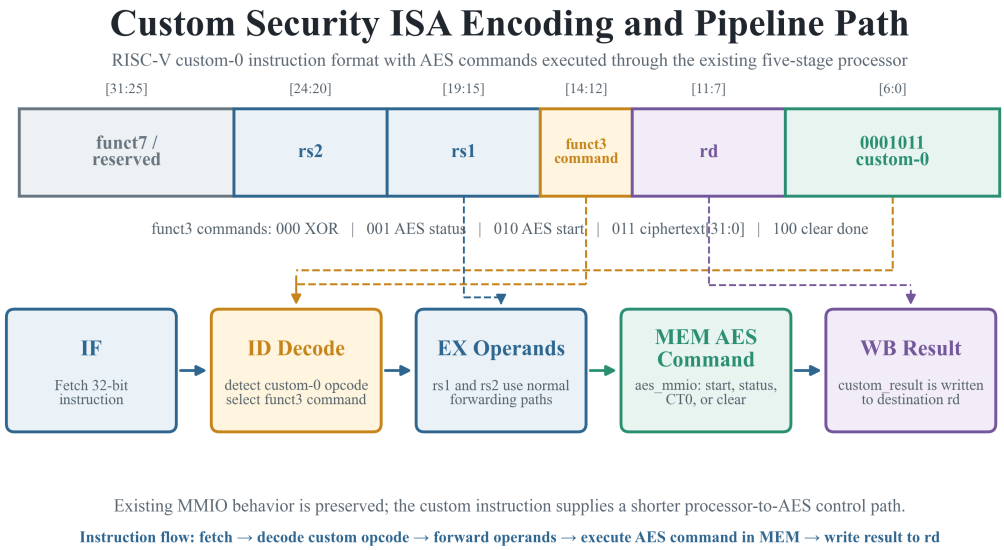


Figure 4.2: Custom-0 security-instruction encoding and execution path. All commands use opcode `0001011`; `funct3` selects XOR, AES status, AES start, ciphertext-word read, or done clear. The IF stage fetches the 32-bit word, ID identifies the custom opcode and command, EX forwards `rs1` and `rs2`, MEM sends the operation to `aes_mmio`, and WB returns `custom_result` to `rd`. The existing MMIO path remains available for software portability and detailed register-level control.

CHAPTER 5:

RTL IMPLEMENTATION

The RTL is modular. Processor files implement the five-stage CPU, while peripheral files implement AES, UART, sensor/SPI, DMA-lite, interrupt, and power management. This organization is useful for thesis work because each block can be explained and verified independently before the full SoC scenario is run [1, 4].

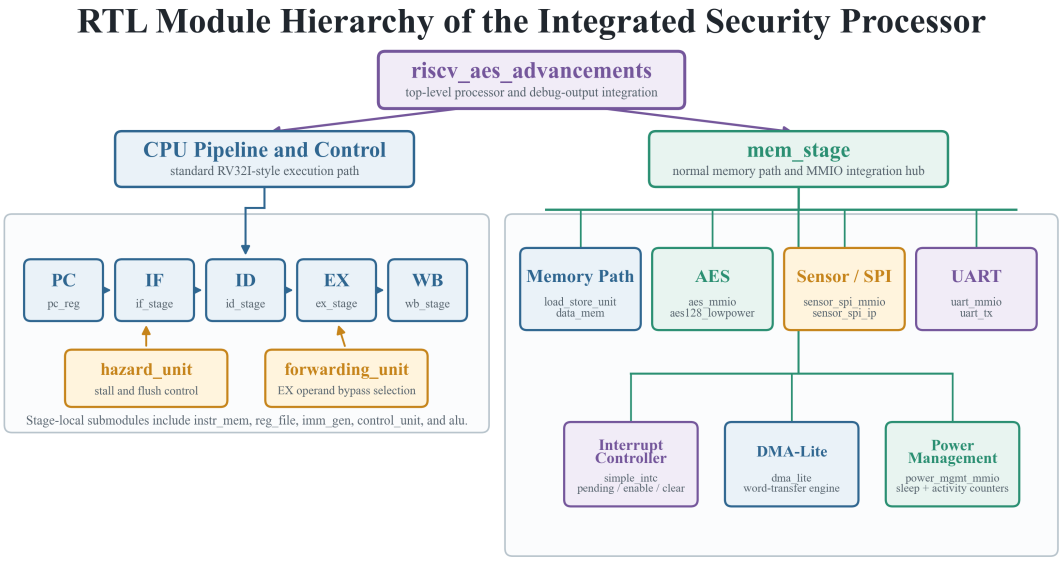


Figure 5.1: RTL hierarchy of the integrated processor. The top-level `riscv_aes_advancements` module connects the program counter, IF, ID, EX, MEM, and WB stages with the hazard and forwarding units. Stage-local logic includes instruction memory, register file, immediate generator, control unit, and ALU. The MEM-stage branch contains the load/store and data-memory path together with `aes_mmio`/`aes128_lowpower`, sensor SPI, UART, interrupt controller, DMA-lite, and power-management modules. This hierarchy makes the MEM stage the single internal address-decoding and peripheral-integration point.

Important top-level observability signals include `current_pc_debug`, `aes_done_debug`, `aes_ciphertext_debug`, `uart_tx`, `irq_debug`, `sleep_debug`, and `activity_counter_debug`. These outputs are useful in simulation and FPGA analysis because they prevent the design from becoming a closed black box and give the fitter externally observable logic to preserve [1, 18].

Table 5.1: Important system-level signals used in verification

Signal	Width/Type	Purpose
`clk`, `rst_n`	1 bit	System clock and active-low reset
`current_pc_debug`	32 bit	Observes instruction flow through the CPU
`aes_done_debug`	1 bit	Indicates AES completion
`aes_ciphertext_debug`	32 bit	Exposes ciphertext word for debug
`uart_tx`	1 bit	Serial encrypted-data output
`irq_debug`	1 bit	Combined interrupt observation
`sleep_debug`	1 bit	Low-power sleep request observation
`activity_counter_debug`	32 bit	Exposes selected activity counter state

The MEM stage is the major integration point. Normal RAM accesses continue through the data memory path, while addresses in the MMIO ranges are routed to AES, sensor/SPI, UART, interrupt, DMA, or power-management registers. Only one selected peripheral drives read data for a given address range, and unmapped regions return a defined default value [4, 12].

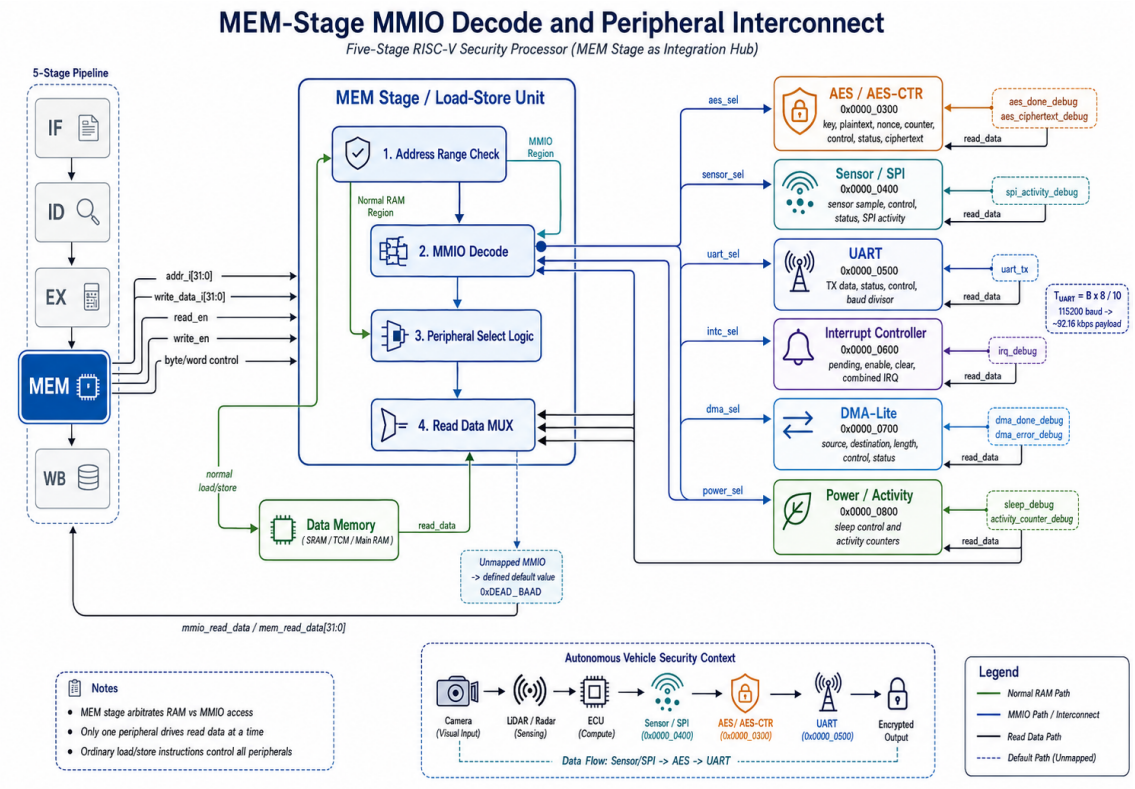


Figure 5.2: MEM-stage MMIO address decoding and peripheral interconnect. Normal RAM accesses remain on the data-memory path, while mapped transactions select one security peripheral and return data through a controlled read multiplexer.

CHAPTER 6:

VERIFICATION METHODOLOGY

The verification strategy is layered. AES known-answer tests first prove the cryptographic primitive. Directed CPU tests then prove the supported instruction behavior. Directed peripheral tests prove MMIO blocks and custom commands. Random smoke tests vary data and addresses. UVM end-to-end testing compares RTL against an independent C model, while functional coverage measures randomized-space exploration. A final scenario-based test executes the complete IoT-style transaction [1, 2].

Table 6.1: Verification layers used in the project

Layer	Evidence	What it proves
AES known-answer	Questa waveform and transcript	Correct AES-128 ECB output after MMIO integration
Directed CPU	Questa waveform and transcript	R, I, load/store, branch, jump, system, fence, pseudo instructions
Directed peripherals	Questa waveform and transcript	AES-CTR, sensor, SPI, UART, INTC, DMA, power, custom ISA
Random smoke	Questa waveform and transcript	Changing values do not only pass fixed constants
UVM end-to-end	UVM waveform, C-DPI reference, scoreboard transcript	RTL AES-CTR and UART line match independent reference
Functional coverage	100-transaction waveform and transcript	114/114 portable bins covered, 100 UART matches
Full SoC scenario	Questa waveform and transcript	Sensor to DMA to AES-CTR to UART to interrupt to sleep

The UVM environment verifies the cryptographic and serial-output path against an independently compiled C model rather than relying only on internal RTL observations [2, 20].

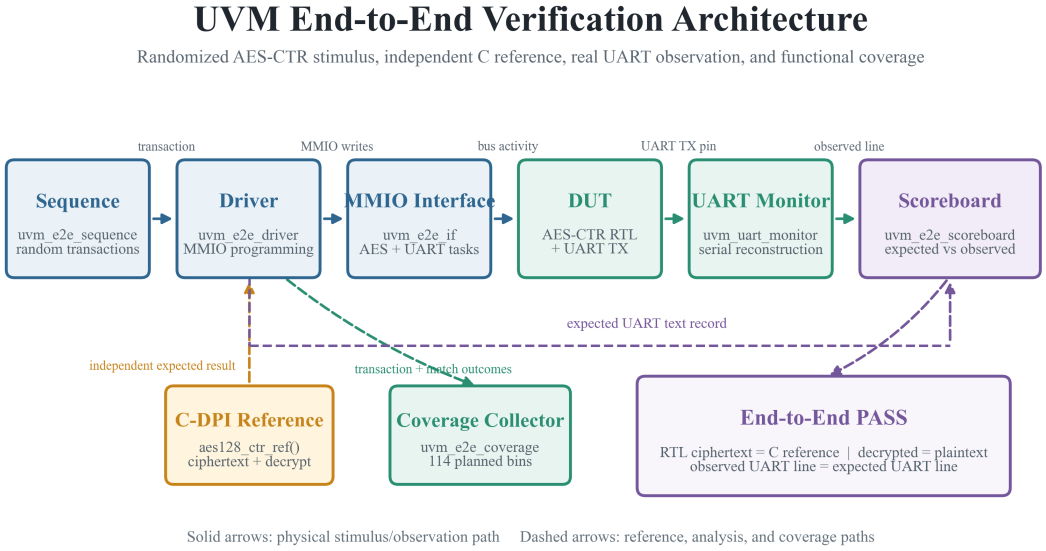


Figure 6.1: UVM end-to-end verification architecture. A deterministic randomized sequence produces plaintext, key, nonce, and counter transactions. The driver calls the independent C-DPI AES-CTR model, programs the RTL through MMIO, waits for completion, and transmits the resulting text record through the physical UART TX pin. The passive UART monitor reconstructs newline-terminated serial records, while the scoreboard compares them against the expected lines. A separate coverage collector samples value distributions, crosses, RTL/reference ciphertext equality, and decrypted/plaintext equality for a total of 114 planned bins.

Native SystemVerilog covergroups are present under a compile-time option for simulators with the required verification license. The available Questa Intel FPGA Starter Edition flow uses the portable collector, which measures the same planned bins through counters visible in the UVM interface. The final 100-transaction run reports 114 out of 114 portable bins covered and 100 UART matches with zero UVM errors [1, 2].

CHAPTER 7:

WAVEFORM RESULTS AND SELF-CHECKING SUMMARY

This chapter uses actual Questa waveform screenshots captured from the project directory. To keep the report concise, each waveform is followed by a short verification conclusion rather than a long transcript block. A single representative transcript excerpt is included at the end of the chapter to show that the testbench is self-checking and prints observed values, expected values, and pass/fail status [1, 2].

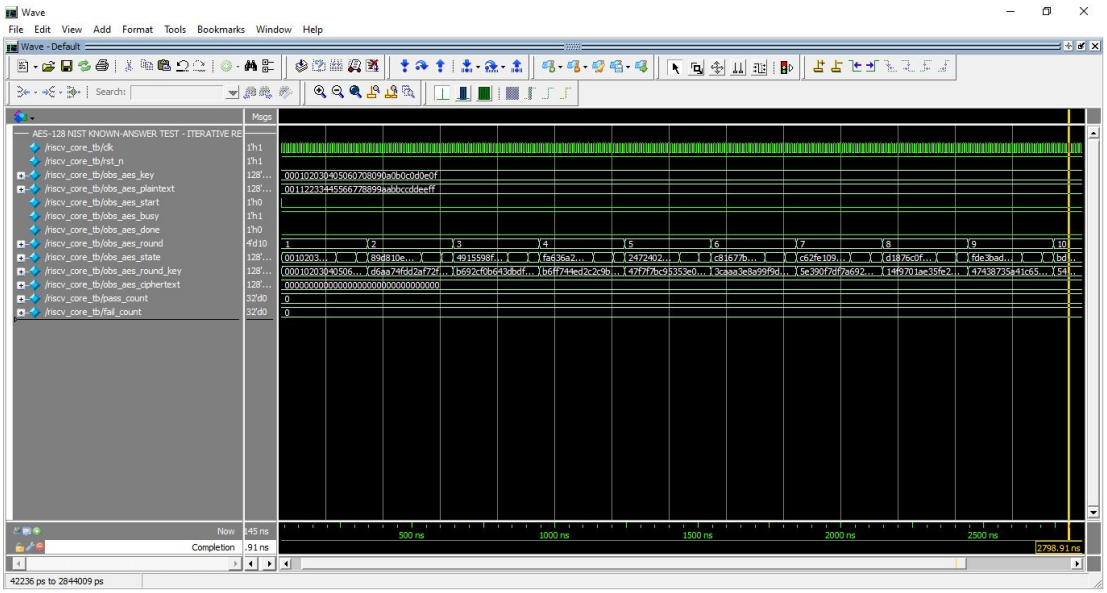


Figure 7.1: AES-128 known-answer MMIO test.

Waveform interpretation. The waveform shows reset release, MMIO loading of the NIST key and plaintext, start assertion, the internal AES round counter advancing, and the done flag at completion. The ciphertext bus settles to the expected AES-128 known-answer value, split here for portrait readability: `69C4E0D86A7B0430` and `D8CDB78070B4C55A`. This verifies the reusable AES primitive after integration behind the MMIO wrapper [2, 30].

Verification conclusion. Test passed with 6 AES known-answer MMIO checks and zero failures. The representative transcript excerpt later in this chapter shows the self-checking `got/expected/PASS` format used by the testbench.

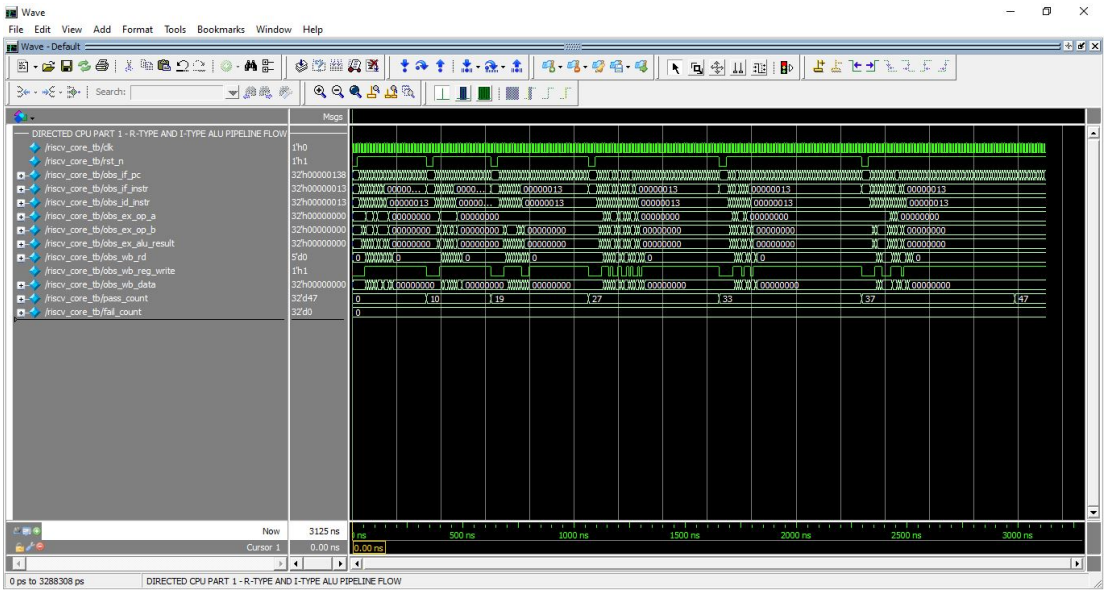


Figure 7.2: Directed R-type and I-type processor execution.

Waveform interpretation. The first directed CPU waveform captures register-register and immediate instructions through the five-stage flow. The PC advances normally, the decode and execute controls select ALU operands, and the writeback path returns the expected values to the register file. The transcript confirms ADD, SUB, shift, comparison, logical, and immediate instructions [1, 8].

Verification conclusion. Test passed with the directed CPU instruction checks reporting zero failures. The same self-checking task format compares observed register or memory values against expected values for each instruction group.

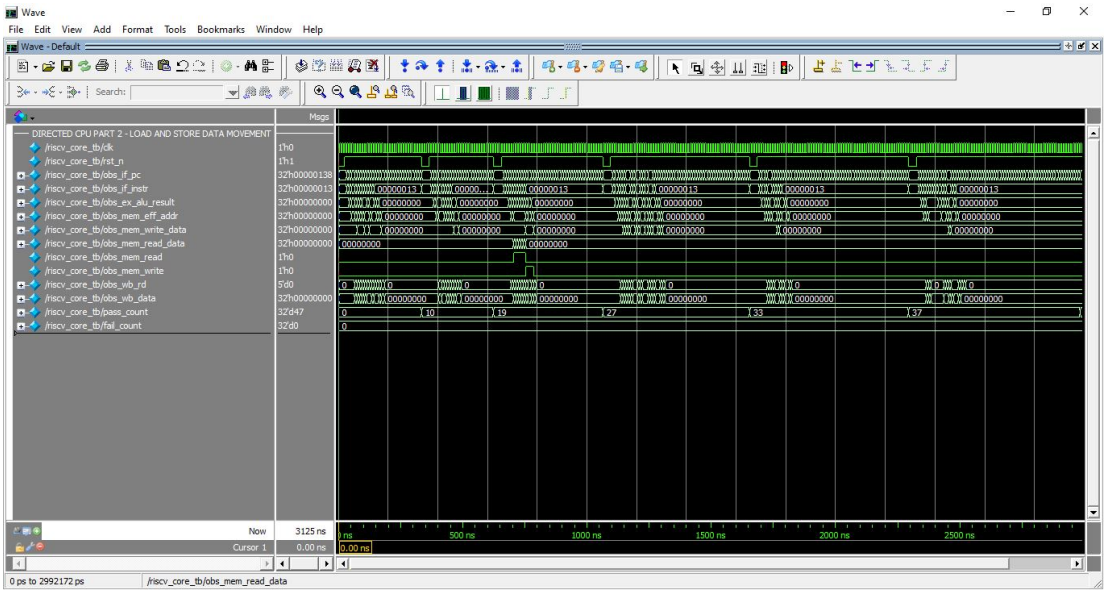


Figure 7.3: Directed load and store processor execution.

Waveform interpretation. The load-store waveform focuses on address generation, data memory selection, byte-enable behavior, sign extension, zero extension, and writeback. LB, LH, LW, LBU, LHU, SB, SH, and SW are checked against known memory words, so the memory stage is verified before MMIO peripherals are considered [1, 12].

Verification conclusion. Test passed with the directed CPU instruction checks reporting zero failures. The same self-checking task format compares observed register or memory values against expected values for each instruction group.

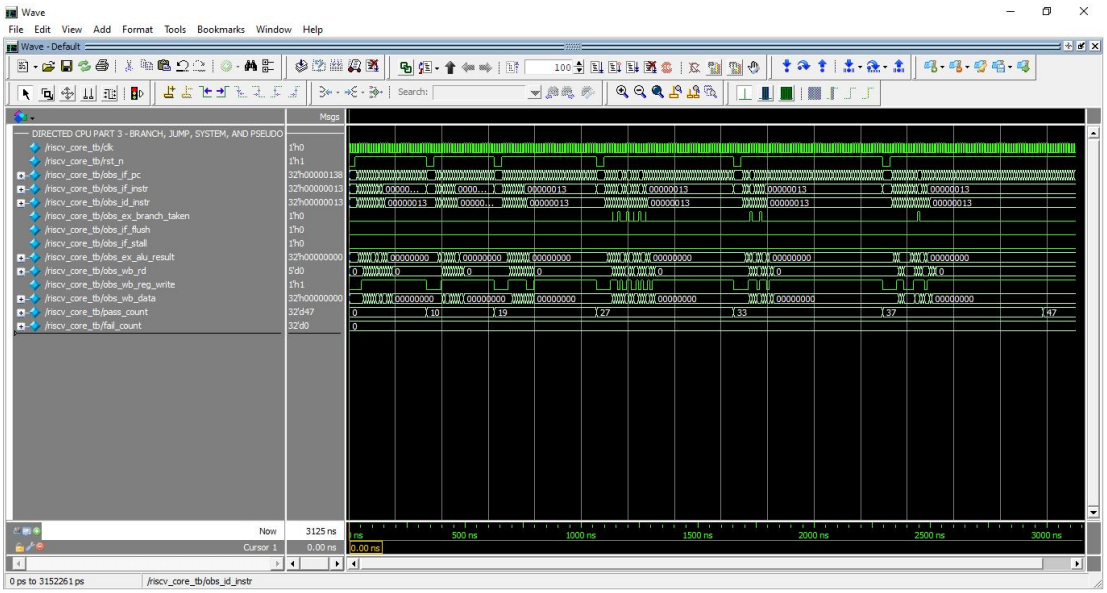


Figure 7.4: Directed branch and jump processor execution.

Waveform interpretation. The branch and jump waveform shows control-flow redirection, pipeline flush behavior, and return-address generation for JAL and JALR. The branch tests deliberately include taken and not-taken cases so that both PC+4 and branch target paths are exercised [8, 12].

Verification conclusion. Test passed with the directed CPU instruction checks reporting zero failures. The same self-checking task format compares observed register or memory values against expected values for each instruction group.

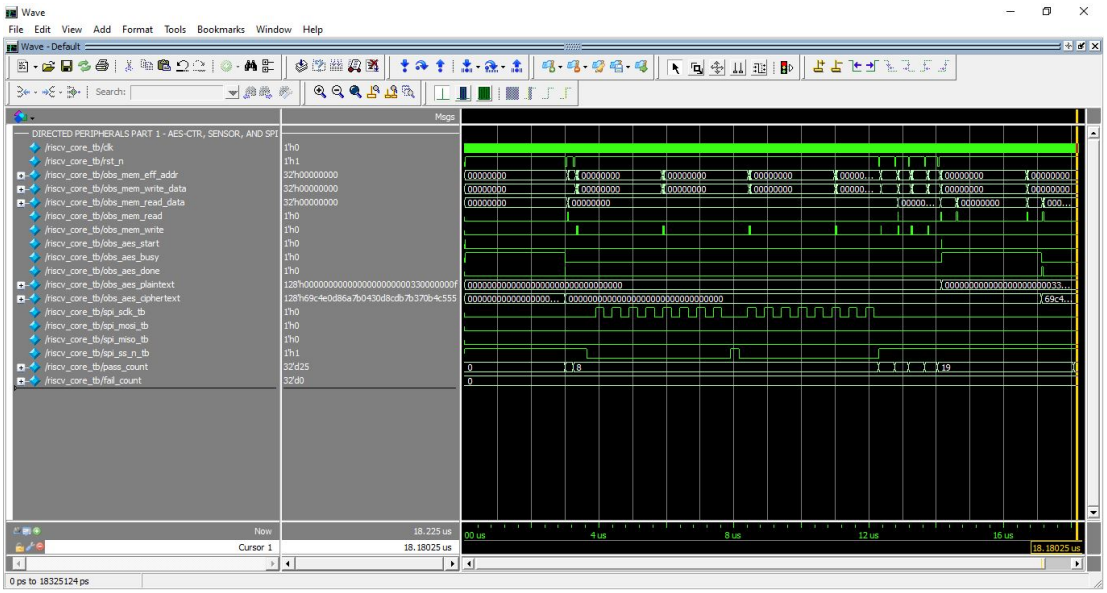


Figure 7.5: AES-CTR, sensor MMIO, and SPI activity.

Waveform interpretation. This waveform verifies the data-input side of the IoT path. Sensor data is read through MMIO, SPI clock activity is observed, and AES-CTR mode is exercised with zero plaintext so that the ciphertext equals the generated keystream. Counter auto-increment is also checked [4, 30].

Verification conclusion. Test passed with the directed peripheral and custom-security checks reporting zero failures. The checks cover AES-CTR, sensor/SPI access, UART status, interrupt pending state, DMA completion, power counters, and custom instruction behavior.

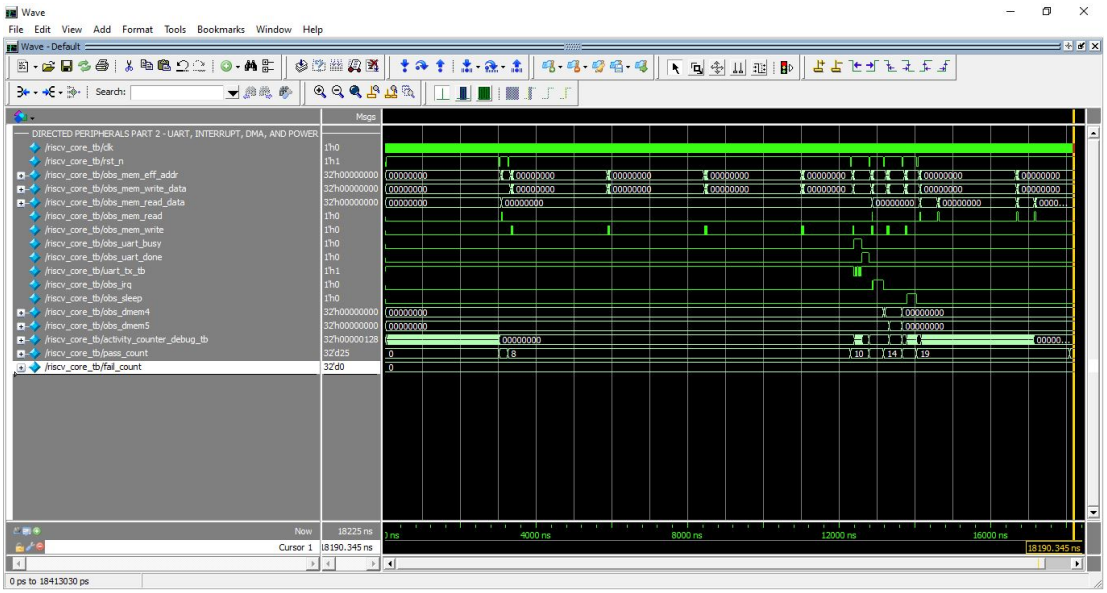


Figure 7.6: UART, interrupt, DMA-lite, and power-management activity.

Waveform interpretation. The second peripheral waveform confirms that a CPU store can start UART transmission, that busy eventually deasserts, that the interrupt controller captures pending bits, that DMA-lite copies a word, and that sleep and activity counters are observable through MMIO [4, 18].

Verification conclusion. Test passed with the directed peripheral and custom-security checks reporting zero failures. The checks cover AES-CTR, sensor/SPI access, UART status, interrupt pending state, DMA completion, power counters, and custom instruction behavior.

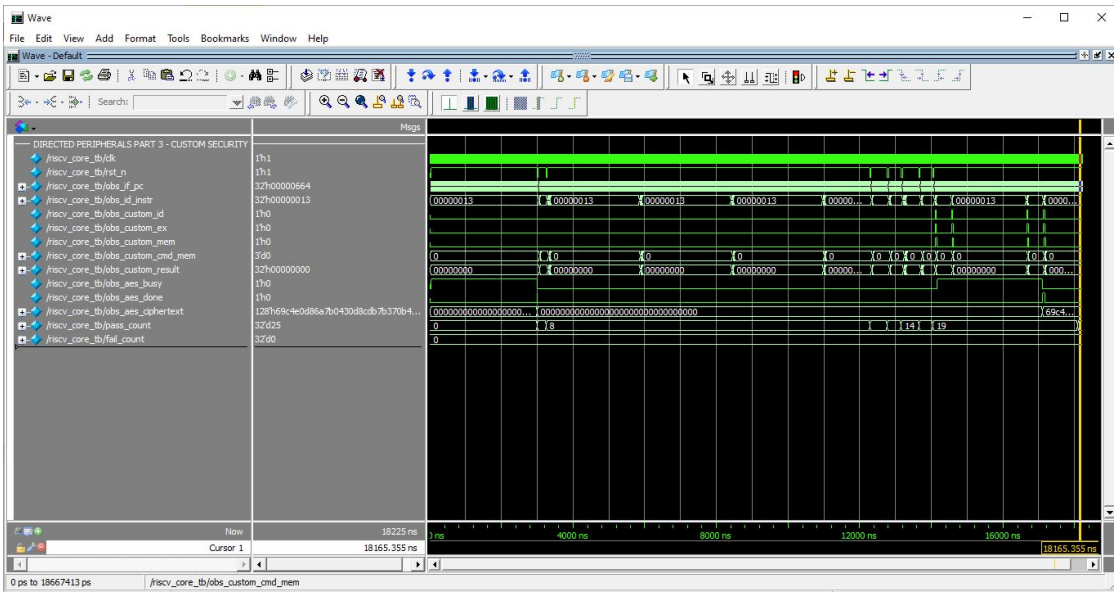


Figure 7.7: Custom security ISA and AES interaction.

Waveform interpretation. The custom instruction waveform verifies the custom-0 security path. The processor decodes custom operations, starts AES-CTR, reads AES status and ciphertext, and clears the completion state without removing the existing MMIO path [8, 12].

Verification conclusion. Test passed with the directed peripheral and custom-security checks reporting zero failures. The checks cover AES-CTR, sensor/SPI access, UART status, interrupt pending state, DMA completion, power counters, and custom instruction behavior.

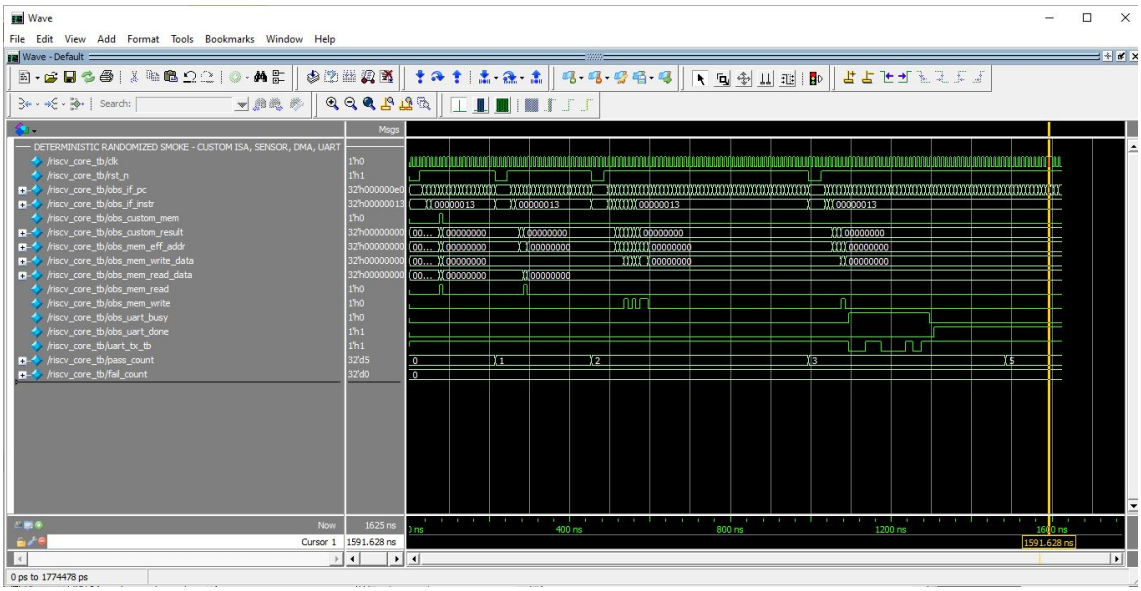


Figure 7.8: Deterministic randomized smoke verification.

Waveform interpretation. The random smoke waveform changes custom-ISA operands, sensor samples, DMA source values, and UART bytes. It is not a replacement for full constrained random verification, but it gives fast confidence that the design does not only pass fixed constants [1, 2].

Verification conclusion. Test passed with 5 deterministic randomized smoke checks and zero failures, confirming that the design also passes changing values rather than only fixed constants.

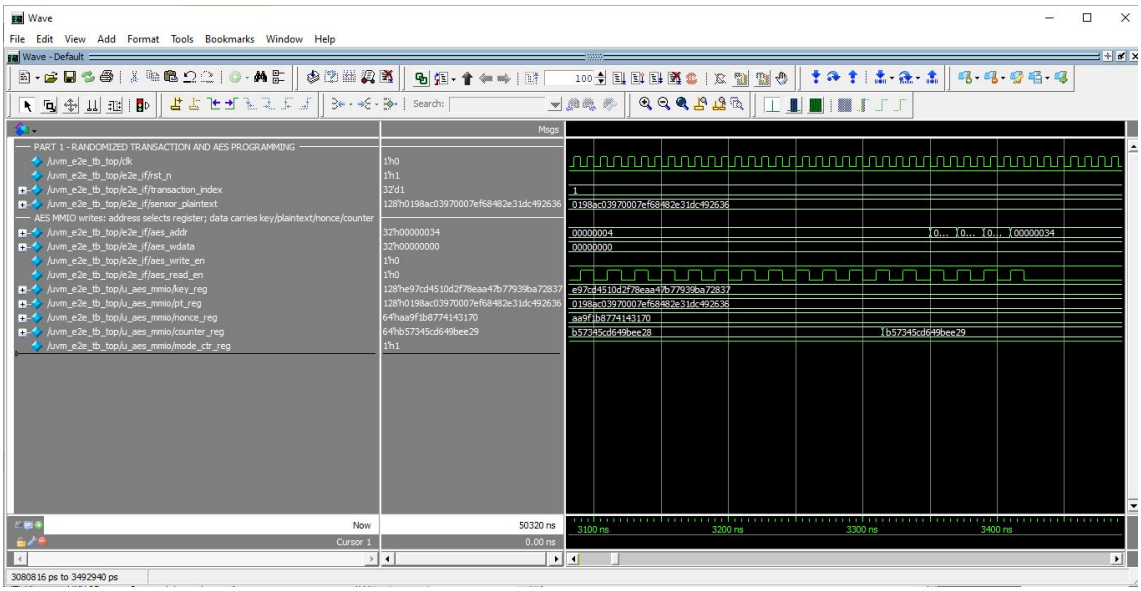


Figure 7.9: UVM randomized inputs and AES-CTR programming.

Waveform interpretation. The first UVM waveform shows the generated plaintext, key, nonce, and counter being driven into the RTL. The UVM driver writes the AES MMIO registers and prepares the expected UART text line using the independent C-DPI reference model [2, 20].

Verification conclusion. Test passed with UVM scoreboard matches and zero UVM errors or fatal messages. The RTL AES-CTR result, decrypted plaintext, and reconstructed UART line matched the independent C-DPI reference result.

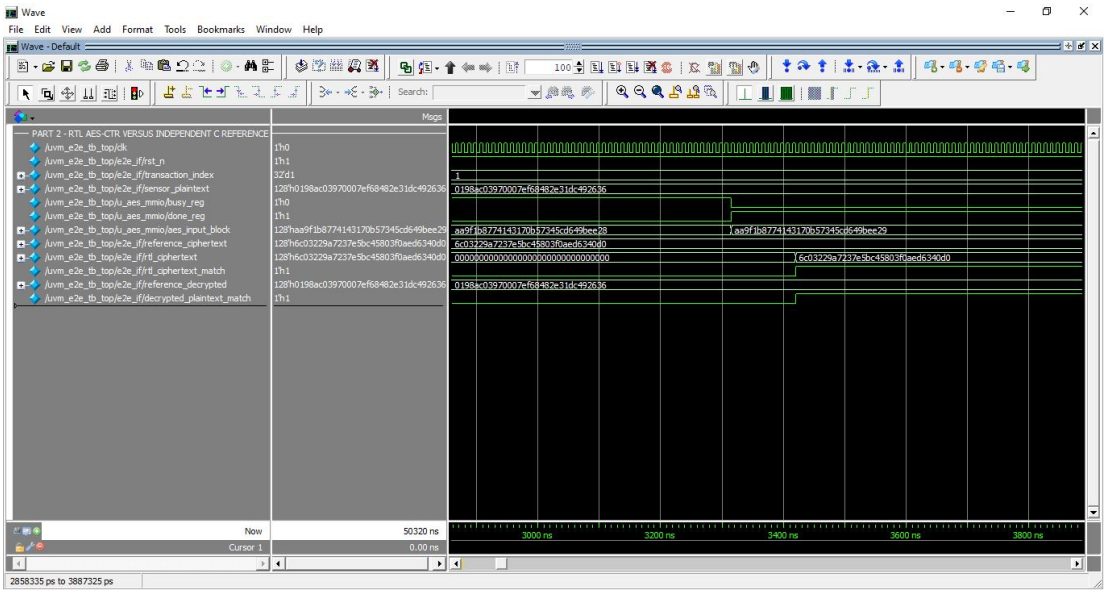


Figure 7.10: RTL AES-CTR ciphertext compared with C reference and decrypt match.

Waveform interpretation. The second UVM waveform is the central end-to-end cryptographic check. RTL ciphertext is compared with the C reference ciphertext, and CTR decryption is applied to prove that the recovered plaintext matches the original randomized input [2, 30].

Verification conclusion. Test passed with UVM scoreboard matches and zero UVM errors or fatal messages. The RTL AES-CTR result, decrypted plaintext, and reconstructed UART line matched the independent C-DPI reference result.

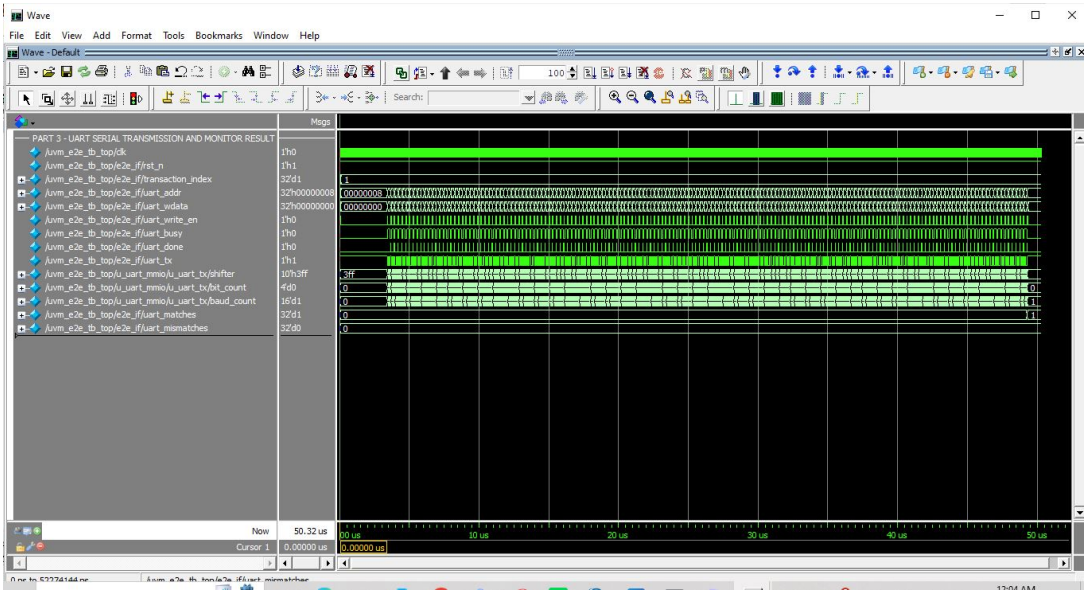


Figure 7.11: UART serial transmission and monitor reconstruction.

Waveform interpretation. The UART waveform shows the serialized text record leaving the RTL UART transmitter. The monitor reconstructs the serial byte stream and forwards the observed line to the scoreboard, which avoids relying only on internal parallel buses [2, 4].

Verification conclusion. Test passed with UVM scoreboard matches and zero UVM errors or fatal messages. The RTL AES-CTR result, decrypted plaintext, and reconstructed UART line matched the independent C-DPI reference result.

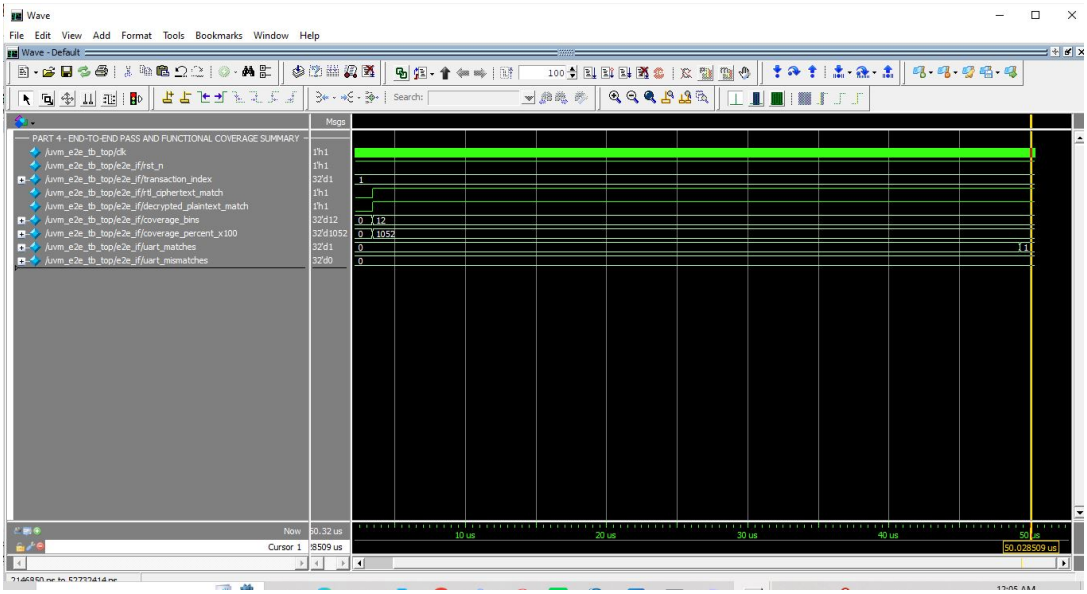


Figure 7.12: UVM scoreboard match flags and coverage summary.

Waveform interpretation. The scoreboard waveform shows final match flags for input, ciphertext, decrypt, and UART reconstruction. The transcript line reports INPUT, KEY, CIPHER, DECRYPTED, and MATCH=PASS, making the evidence readable even without decoding every serial bit [1, 2].

Verification conclusion. Test passed with UVM scoreboard matches and zero UVM errors or fatal messages. The RTL AES-CTR result, decrypted plaintext, and reconstructed UART line matched the independent C-DPI reference result.

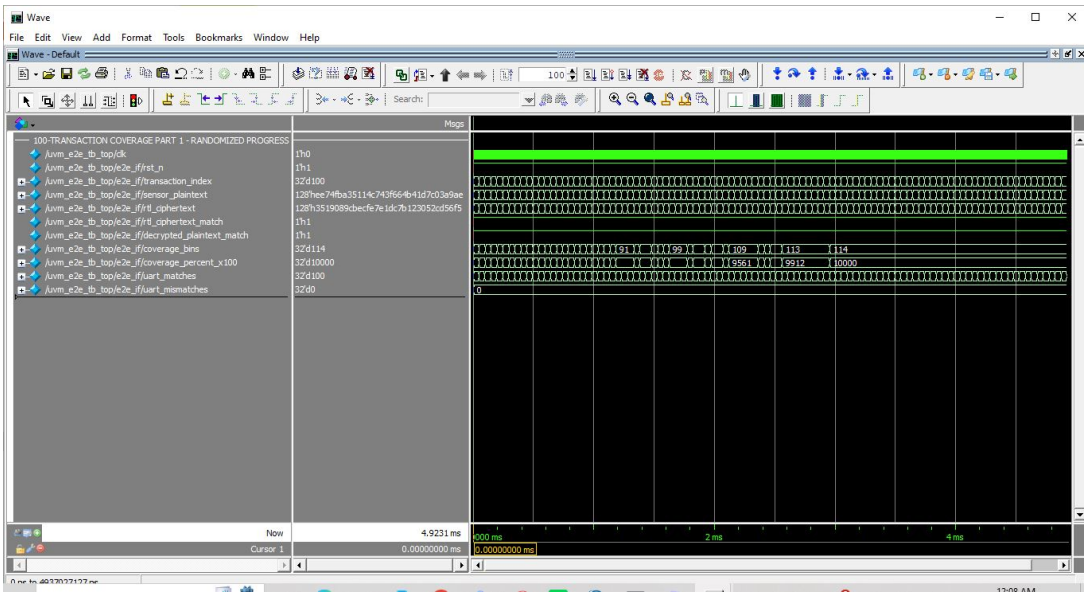


Figure 7.13: Functional coverage transaction progression and bin growth.

Waveform interpretation. The coverage-progress waveform shows the randomized transaction count increasing and the portable coverage bins accumulating. This confirms that coverage is sampled from actual randomized transactions rather than from static initialization [1, 2].

Verification conclusion. Test passed with 100 randomized UART matches, zero UVM errors, zero UVM fatal messages, and 114/114 portable functional-coverage bins covered.

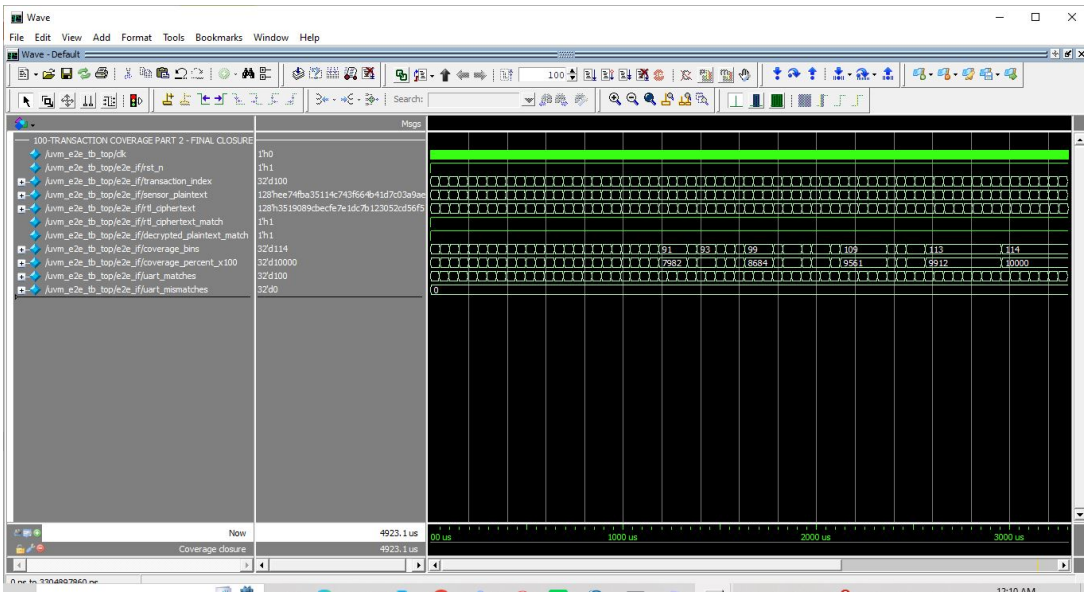


Figure 7.14: Final 100-transaction functional coverage closure.

Waveform interpretation. The closure waveform captures the end of the 100-transaction UVM run. The transcript reports 100 UART matches, zero UVM errors, zero UVM fatal messages, and 114 out of 114 portable coverage bins covered [1, 2].

Verification conclusion. Test passed with 100 randomized UART matches, zero UVM errors, zero UVM fatal messages, and 114/114 portable functional-coverage bins covered.

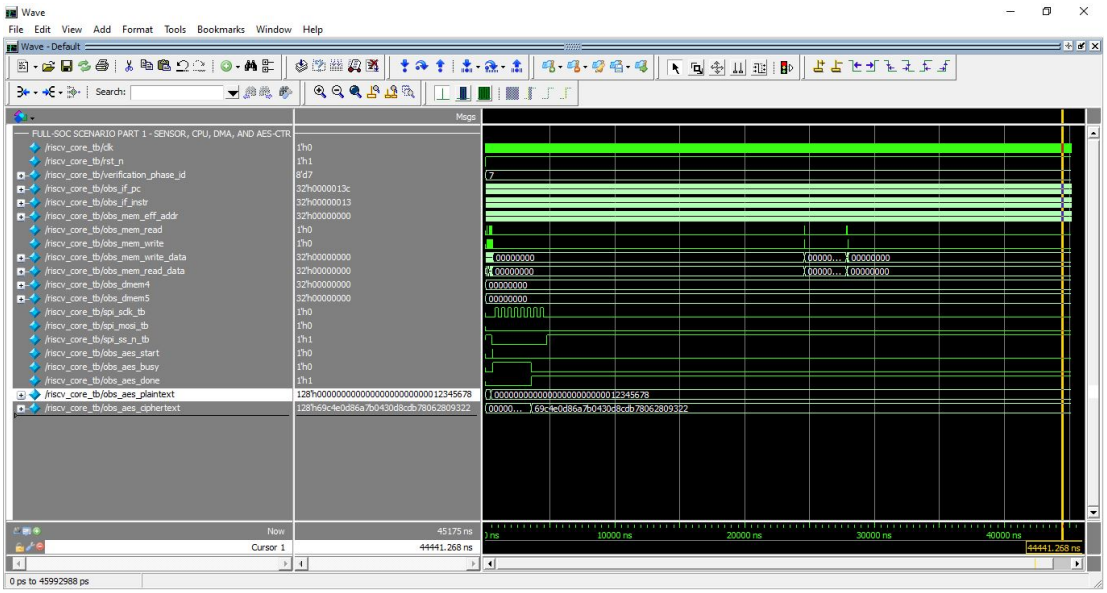


Figure 7.15: Full-SoC sensor, DMA, and AES-CTR data-security path.

Waveform interpretation. The full-SoC datapath waveform follows a realistic transaction: the processor reads the sensor sample, stores it, DMA copies it, AES-CTR encrypts it, and the ciphertext word is checked. This test proves the blocks operate together rather than only as isolated peripherals [4, 5].

Verification conclusion. Test passed with 15 full-SoC scenario checks and zero failures, covering the sensor-to-DMA-to-AES-CTR-to-UART path, interrupt state, activity counters, and sleep request.

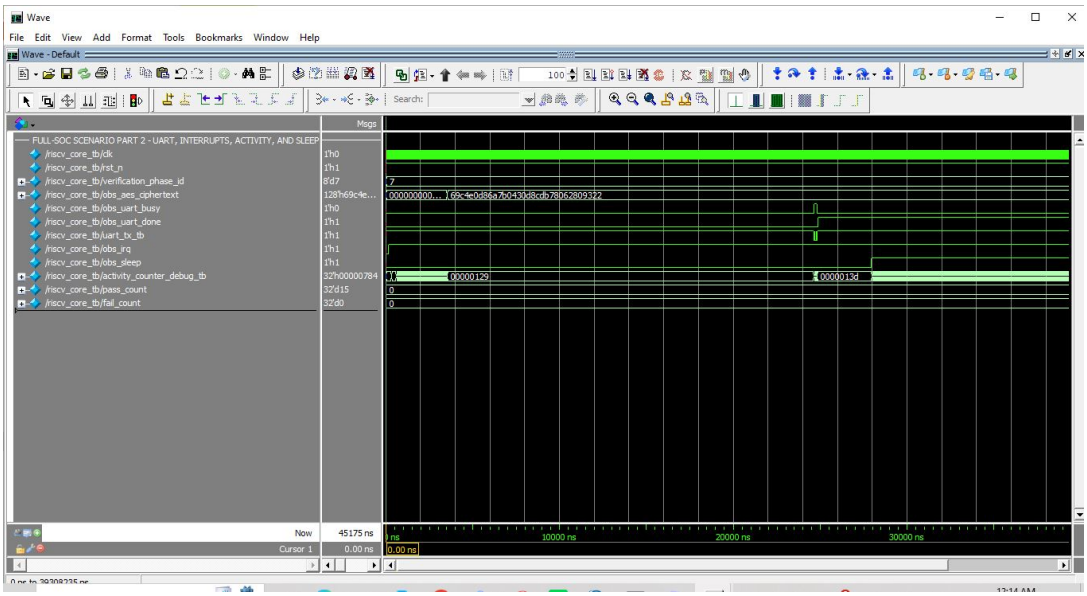


Figure 7.16: Full-SoC UART, interrupt, activity counter, and sleep behavior.

Waveform interpretation. The final scenario waveform verifies the output and control side of the system. UART accepts the ciphertext byte, interrupt pending bits capture AES, UART, sensor, and DMA events, activity counters become nonzero, and the processor requests sleep after transmission [4, 18].

Verification conclusion. Test passed with 15 full-SoC scenario checks and zero failures, covering the sensor-to-DMA-to-AES-CTR-to-UART path, interrupt state, activity counters, and sleep request.

The integrated transcript stored in `output_files/transcript.txt` captures the broader regression where CPU instruction tests, AES ECB, AES-CTR, end-to-end UART print/decrypt, peripheral tests, signal activity, randomized smoke, and full-SoC scenario checks are run together. The final integrated summary reports 107 passes and zero failures. The separately reported UVM end-to-end run reports 25 UART/crypto matches with zero mismatches, and the coverage run reports 100 randomized UART matches with 114/114 portable bins covered [1, 2].

Representative self-checking transcript excerpt

The excerpt below is kept as one compact example of the print style used throughout the verification environment. Each line compares observed RTL output with the expected value and prints `PASS` only when the comparison succeeds.

```
# === AES-128 NIST MMIO tests ===
# [AES] DONE: iterative reusable AES completed -> got=0x00000001 expected=0x00000001 -> PASS
# [AES] BUSY: busy deasserted after completion -> got=0x00000000 expected=0x00000000 -> PASS
# [AES] CT0: ciphertext[31:0] -> got=0x70b4c55a expected=0x70b4c55a -> PASS
# [AES] CT1: ciphertext[63:32] -> got=0xd8cdb780 expected=0xd8cdb780 -> PASS
# [AES] CT2: ciphertext[95:64] -> got=0x6a7b0430 expected=0x6a7b0430 -> PASS
# [AES] CT3: ciphertext[127:96] -> got=0x69c4e0d8 expected=0x69c4e0d8 -> PASS
# ================================================
# RV32I-style directed verification summary: PASS=6 FAIL=0
# Lightweight IoT Security Processor + Custom ISA verification summary: PASS=6 FAIL=0
# ================================================
# ALL TESTS PASSED
# Errors: 0, Warnings: 1
```

CHAPTER 8:

SYNTHESIS, TIMING, POWER, AND NETLIST RESULTS

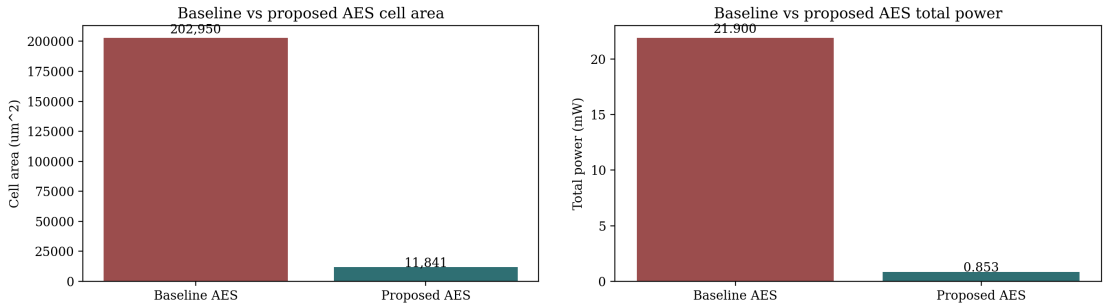
This chapter contains three independent result domains. They are not combined because they represent different design scopes, target technologies, and research questions. The first domain compares only the baseline AES-128 architecture and the proposed iterative hardware-reusable AES-128 architecture using Cadence Genus and a 55 nm standard-cell library. The second domain evaluates the complete AES plus RISC-V security processor using Questa for RTL verification and Quartus for FPGA implementation. The third domain maps the complete SoC with Cadence Genus to GPDK045 HVT cells at a slow operating corner. This separation prevents FPGA resources, standalone AES results, and complete-SoC ASIC-style estimates from being presented as if they were directly interchangeable [3, 5].

8.1 Result Domain I: Standalone AES Comparison Using Cadence Genus at 55 nm

The purpose of this result domain is to quantify the architectural benefit of AES datapath reuse. Both the baseline AES and proposed low-power AES are synthesized under the same Cadence Genus flow using the TSMC 55 nm low-power RVT standard-cell library at the typical process corner, 1.0 V supply, and 25 °C. Therefore, the cell count, cell area, internal power, switching power, leakage power, and timing values in this section belong only to the standalone AES comparison. They do not include the RISC-V processor, MMIO peripherals, UART, SPI, DMA, interrupt controller, or activity counters.

Table 8.1: Standalone proposed AES-128 Cadence Genus metrics at 55 nm

Metric	Value	Evidence source
Cadence Genus cell count	2,694	GCON result, `report_area.rpt`
Cadence Genus cell area	11,841.480 μm^2	`report_area.rpt`
Cadence Genus total power	8.53125×10^{-4} W	`report_power.rpt`
Genus internal power	6.96687×10^{-4} W	`report_power.rpt`
Genus switching power	1.55986×10^{-4} W	`report_power.rpt`
Genus leakage power	4.51803×10^{-7} W	`report_power.rpt`
Critical data path	3.102 ns	`report_timing.rpt`
Setup slack at 10 ns target	6.806 ns	`report_timing.rpt`



(a) AES area comparison.

(b) AES power comparison.

Figure 8.1: Cadence Genus 55 nm comparison of baseline AES and proposed iterative hardware-reusable AES.

The 55 nm Cadence Genus comparison shows why hardware reuse is valuable. The proposed AES reduces cell count from 83,352 to 2,694 and cell area from 202,949.64 μm^2 to 11,841.48 μm^2. Total power reduces from 21.9 mW to approximately 0.853 mW under the same ASIC synthesis conditions. The tradeoff is that encryption is iterative, so latency is higher than a fully unrolled design. For sensor telemetry, this is acceptable because the communication and sampling intervals are usually longer than the AES block-processing interval [26, 27].

Table 8.2: Baseline AES versus proposed reusable AES using Cadence Genus at 55 nm

Metric	Baseline AES	Proposed AES	Note
Cell count	83,352	2,694	96.77% fewer cells
Cell area (μm^2)	202,949.64	11,841.48	94.17% lower area
Total power (W)	2.19×10^{-2}	8.53125×10^{-4}	96.10% lower total power
Internal power (W)	1.29×10^{-2}	6.96687×10^{-4}	Lower internal switching due to fewer active blocks
Switching power (W)	8.97×10^{-3}	1.55986×10^{-4}	Lower activity from datapath reuse
Architecture	Parallel/unrolled style	Iterative reusable datapath	Area and power traded against latency

8.2 Result Domain II: Integrated AES Plus RISC-V Processor Using Questa and Quartus

The integrated-system results answer a different question: whether the reusable AES accelerator functions correctly when connected to the five-stage processor and its peripherals, and whether the complete RTL can be implemented on an FPGA. Questa verifies processor instructions, MMIO accesses, AES ECB and CTR modes, custom security instructions, sensor/SPI operation, UART transmission, DMA, interrupt behavior, power counters, randomized transactions, C-DPI reference comparison, functional coverage, and the complete application scenario. The waveform and transcript evidence is presented in Chapter 7. Quartus then synthesizes and fits the same integrated RTL to report FPGA resources, post-fit timing, and the synthesized netlist. No ASIC cell-area or ASIC power value from the standalone 55 nm AES study is used as an integrated-processor result.

8.2.1 Questa Functional Verification of the Integrated System

The integrated Questa regressions confirm that the AES accelerator and processor operate together rather than as isolated modules. Directed CPU tests verify the supported RV32I-style instruction behavior, peripheral tests verify MMIO and custom commands, randomized tests vary data and control values, and the UVM environment compares RTL AES-CTR results against an independent C reference model. The full-SoC scenario executes the sensor-to-UART path and checks DMA completion, interrupt pending state, activity counters, and entry into sleep. These are functional verification results; they are not synthesis, area, timing, or power measurements.

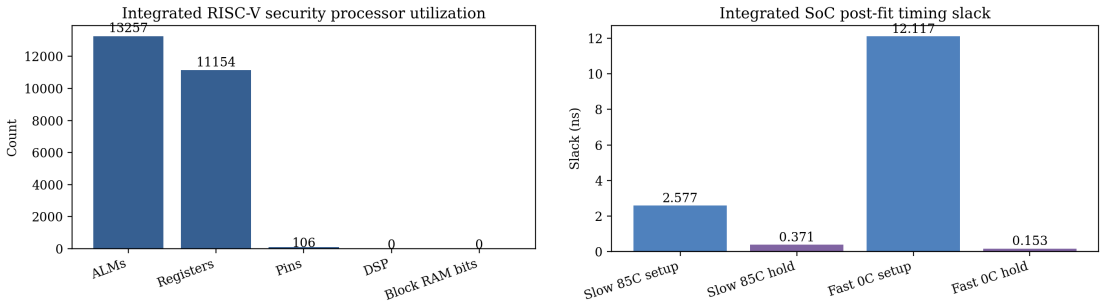
8.2.2 Quartus FPGA Implementation of the Integrated System

The integrated RISC-V security processor is larger than the standalone AES core because it includes the pipeline, memories, MMIO decode, UART, SPI interface, DMA, interrupt, and activity counters. Quartus Prime 23.1std.1 Build 993 SC Lite Edition maps the complete design to a Cyclone V device with 13,257 ALMs, 11,154 registers, 106 pins, and no DSP or block RAM usage. The absence of DSP blocks is expected because AES and RV32I integer operations are logic-based in this design [4, 8].

Table 8.3: Integrated AES plus RISC-V processor Quartus FPGA implementation summary

Metric	Value	Source
Quartus version	23.1std.1 Build 993 SC Lite Edition	Fitter summary
Family and device	Cyclone V, 5CGXFC7C7F23C8	Fitter summary
Logic utilization	13,257 / 56,480 ALMs (23%)	Fitter summary
Registers	11,154	Fitter summary
Pins	106 / 268 (40%)	Fitter summary
Block memory bits	0 / 7,024,640	Fitter summary
DSP blocks	0 / 156	Fitter summary
PLLs	0 / 13	Fitter summary
Fitter status	Successful, 0 errors, 3 warnings	Fitter log

The detailed implementation source is `riscv_aes_advancements.fit.summary`; fitter messages are recorded in `riscv_aes_advancements.fit.log`.



(a) FPGA resource counts.

(b) Post-fit timing slack.

Figure 8.2: Integrated AES plus RISC-V processor Quartus FPGA results.

The post-fit TimeQuest result is timing clean for the 20 ns clock constraint, corresponding to 50 MHz. The worst slow-corner setup slack is 2.577 ns and the worst hold slack is 0.371 ns. The approximate critical path delay under this constraint is therefore

$$D_{crit} \approx 20 - 2.577 = 17.423 \text{ ns}, \tag{8.1}$$

which gives an estimated maximum frequency of

$$F_{max} \approx \frac{1}{17.423\ \text{ns}} = 57.4\ \text{MHz}. \tag{8.2}$$

This estimate is derived from the reported setup slack and should be treated as a report-level approximation rather than a replacement for a dedicated Fmax report [5, 18].

Table 8.4: Integrated AES plus RISC-V processor Quartus post-fit timing summary

Timing check	Slack	TNS
Slow 1100 mV 85C setup	2.577 ns	0.000
Slow 1100 mV 85C hold	0.371 ns	0.000
Slow 1100 mV 0C setup	2.744 ns	0.000
Fast 1100 mV 85C setup	11.043 ns	0.000
Fast 1100 mV 0C setup	12.117 ns	0.000

The completed fitted Cyclone V database was analyzed using Quartus Power Analyzer 23.1 at the 50 MHz clock constraint. The vectorless run reports an average toggle rate of 4.819 million transitions/s and an estimated junction temperature of 28.1 °C at 25 °C ambient. Because no VCD or SAIF activity was supplied, these values are low-confidence implementation estimates rather than measured board power.

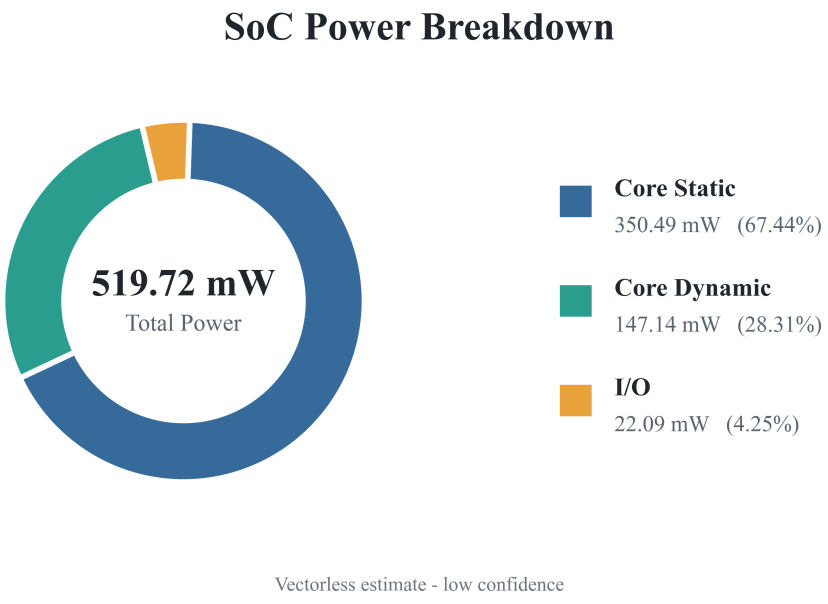


Figure 8.3: Quartus post-fit vectorless power breakdown for the complete processor SoC. The estimated total is 519.72 mW, comprising 350.49 mW core static power (67.44%), 147.14 mW core dynamic power (28.31%), and 22.09 mW I/O power (4.25%). These values use fitted-netlist and delay information but estimated switching activity; therefore the result is explicitly marked low confidence and should not be treated as measured hardware power.

The power split is useful as a first implementation-level indication, but it must be interpreted carefully. Static power contributes approximately 67.44% of the total estimate, core dynamic power contributes 28.31%, and I/O contributes 4.25%. The hierarchy report attributes 4.56 mW of estimated dynamic-plus-routing power to the reusable AES core and 6.97 mW to the complete AES MMIO hierarchy, while data memory and its routing dominate the MEM-stage estimate. A future activity-based run should annotate realistic sensor, processor, AES, DMA, and UART switching from simulation before using these figures for energy-per-transaction claims. Work on dynamic voltage and frequency control also shows that meaningful power management depends on workload and operating-point information rather than a single static estimate [14].

The Quartus synthesized hierarchy places much of the integrated logic inside the memory/peripheral stage because AES, UART, sensor/SPI, DMA, interrupt, and power-management logic are attached there. This is an expected architectural consequence of using the MEM stage as a compact internal MMIO interconnect. The Quartus netlist confirms that the processor pipeline, AES wrapper, iterative AES submodules, UART, sensor interface, DMA, interrupt controller, and power-management logic are retained in the integrated FPGA design [1, 3].

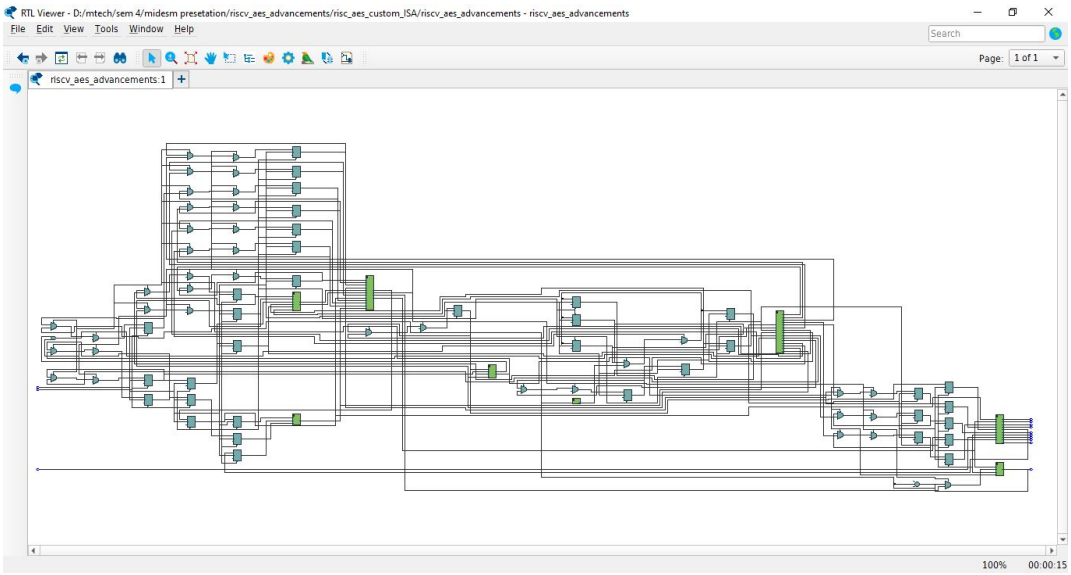


Figure 8.4: Quartus synthesized netlist view of the integrated AES plus RISC-V processor.

Figure 8.5 expands the execute-stage region in the Quartus RTL Viewer. The dense input buses entering the selected hierarchy correspond to decoded control fields, operand values, immediate data, forwarding selections, and pipeline state. Inside the highlighted region, multiplexing and combinational logic select forwarded operands, perform ALU and branch-related operations, and prepare the result and control information passed toward the MEM stage. The output bus structure reflects the parallel transfer of the ALU result, store data, destination-register index, and downstream control signals.

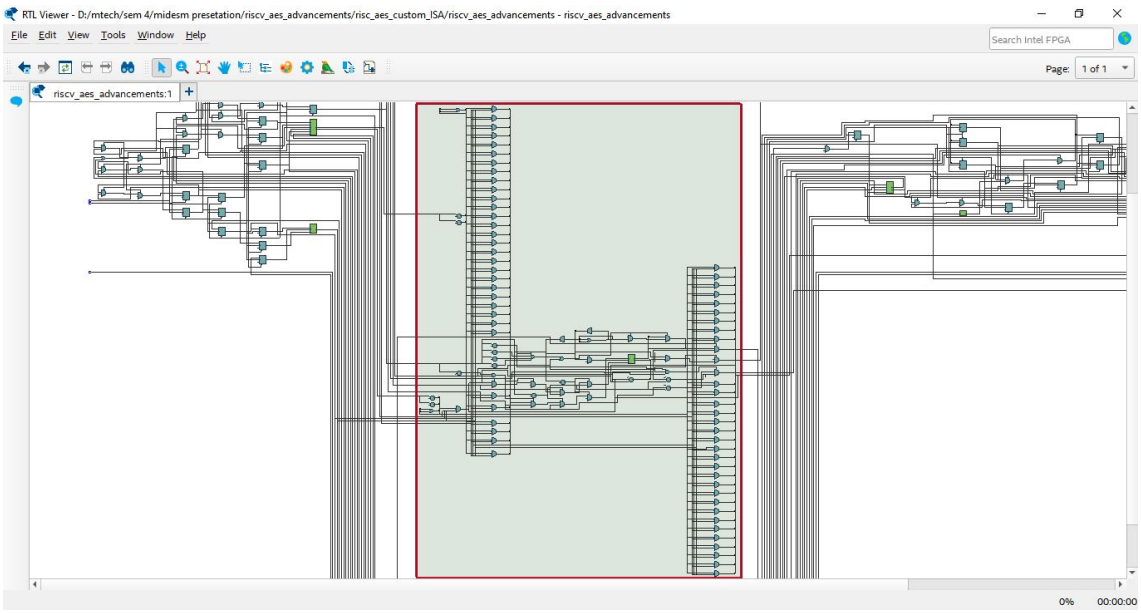


Figure 8.5: Quartus RTL Viewer expansion of the RISC-V execute-stage netlist. The highlighted EX-stage hierarchy contains operand-selection, forwarding, ALU, comparison, and pipeline-output logic.

Figure 8.6 expands the MEM-stage region. Its larger fan-in and fan-out are consistent with the architectural role of this stage as both the normal load/store path and the MMIO integration hub. Address and write-data buses enter the stage together with read, write, transfer-size, and pipeline-control signals. The selected region contains address-range decoding, peripheral-select logic, data-memory access formatting, peripheral control paths, and the read-data multiplexer that returns either RAM data or one selected MMIO response. The wide connections leaving the hierarchy carry memory results, status information, and write-back controls.

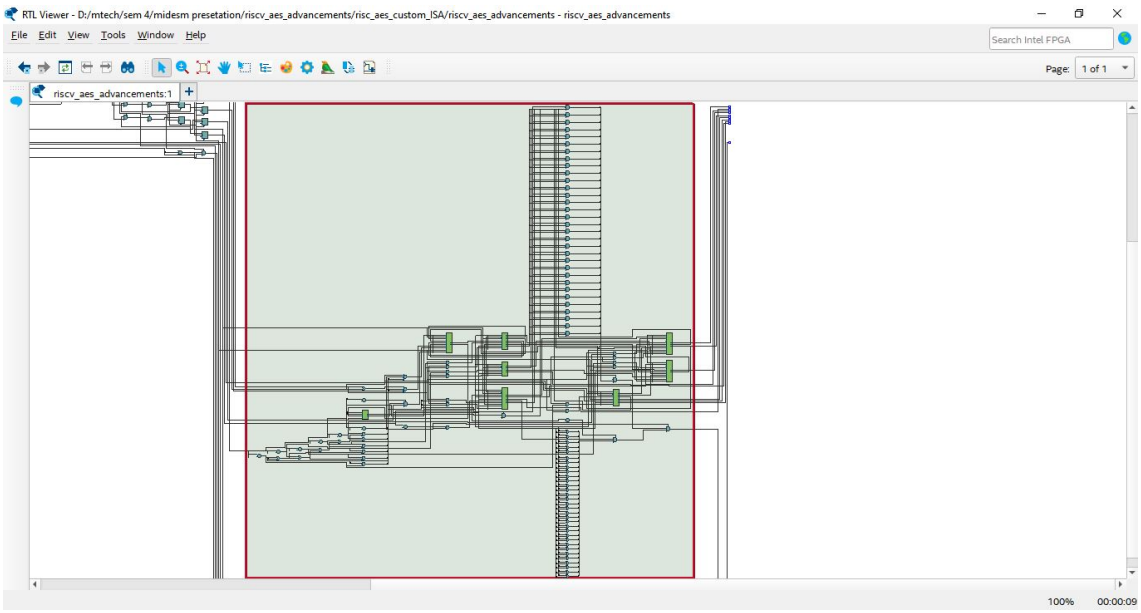


Figure 8.6: Quartus RTL Viewer expansion of the RISC-V memory-stage netlist. The highlighted MEM-stage hierarchy contains normal memory access logic, MMIO decoding, peripheral selection, and read-data routing.

8.3 Result Domain III: Complete SoC Cadence Genus HVT Synthesis

The final ASIC-style experiment synthesizes the complete `riscv_aes_advancements` hierarchy rather than the AES core alone. Cadence Genus 23.14-s090_1 reads the combined SystemVerilog RTL, elaborates the five-stage processor and all retained peripherals, applies the 20 ns all-signals SDC, and maps the design exclusively to the GPDK045 high-threshold-voltage basic-cell library. The selected library is `slow_vdd1v0_basicCells_hvt.lib`; the report records the active characterized operating condition as slow process, 0.9 V and 125 °C. HVT mapping is intended to reduce leakage, while the slow corner gives a conservative setup-timing view [5, 19].

Table 8.5: Complete-SoC Cadence Genus HVT synthesis environment

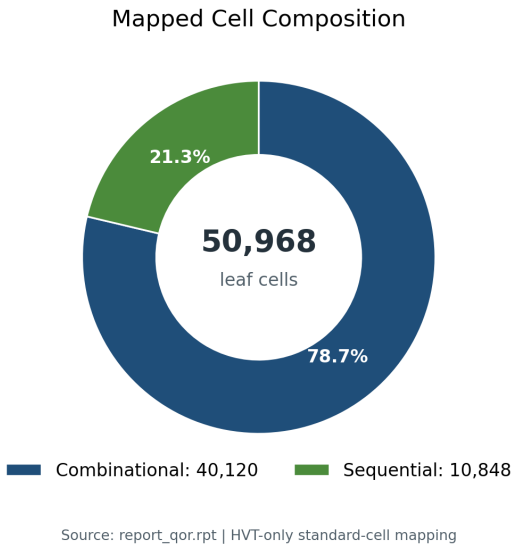
Item	Value	Primary evidence
Synthesis tool	Genus 23.14-s090_1	`risc_aes_finalReports/genus.log`
Elaborated top	`riscv_aes_advancements`	`genus.log`
Technology library	GPDK045 HVT basic cells	`slow_vdd1v0_basicCells_hvt.lib`
Reported operating condition	Slow, 0.9 V, 125 °C	Area and timing report headers
Clock constraint	20.000 ns, 50 MHz	`riscv_aes_genus_all_signals.sdc`
Clock uncertainty	0.200 ns	All-signals SDC
Top-level interface	3 inputs, 103 outputs	All-signals SDC
Synthesis effort	Medium generic, map and optimization	`genus_script.tcl`

8.3.1 Area and Cell Composition

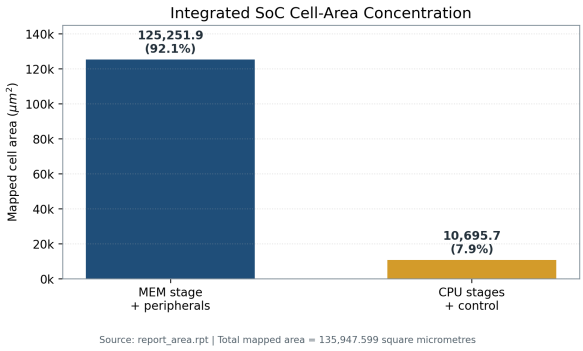
The mapped SoC contains 50,968 leaf cells and occupies 135,947.599 μm^2 of timing-library cell area. Of these cells, 40,120 are combinational and 10,848 are sequential, corresponding to 78.72% and 21.28% respectively. The report records no physical-cell or estimated net area because this is a synthesis-stage result rather than a placed-and-routed layout. The values must therefore be interpreted as mapped standard-cell area, not final silicon die area [3, 25].

Table 8.6: Complete-SoC mapped area and structural composition using GPDK045 HVT cells

Metric	Reported value	Interpretation
Leaf cells	50,968	Total mapped standard-cell instances
Sequential cells	10,848 (21.28%)	Pipeline, memories represented as registers, counters, and peripheral state
Combinational cells	40,120 (78.72%)	ALU, decode, MMIO routing, AES transformations, and control logic
Hierarchical instances	17	Retained named hierarchy after optimization
Mapped cell area	135,947.599 μm^2	Pre-layout timing-library cell area
Physical and net area	0 μm^2	Placement and routing were not performed in this Genus run



(a) Leaf-cell composition.



(b) Named hierarchy area concentration.

Figure 8.7: Complete-SoC structural results extracted from `report_qor.rpt` and `report_area.rpt`. The MEM-stage hierarchy contains 125,251.891 μm^2, or 92.13% of the mapped area, because it contains the data-memory implementation, AES subsystem, sensor/SPI, UART, DMA, interrupt controller, power management, and the MMIO read-data network.

The area hierarchy reinforces an architectural observation already visible in the RTL and Quartus netlist: the MEM stage is more than a pipeline register boundary. It is also the internal peripheral interconnect. Genus attributes 46,876 cells and 125,251.891 μm^2 to this hierarchy, while the remainder of the processor and control logic accounts for approximately 10,695.708 μm^2. This does not mean the processor core itself is negligible; it shows that register-based data storage, AES lookup logic, and peripheral integration dominate this particular synthesizable prototype [4, 8].

8.3.2 Timing Closure

The worst setup path begins at `rd_wb_reg[0]` and terminates at `current_pc_reg[31]`. It traverses control selection, arithmetic and branch-related logic before reaching the program-counter input. At the 20 ns target, Genus reports a 17.169 ns data path, 0.316 ns setup time, 0.200 ns clock uncertainty, and positive setup slack of 2.315 ns. Total negative slack is zero and no violating setup paths are reported [5, 18].

$$T_{\text{effective}} = T_{\text{constraint}} - S_{\text{setup}} = 20.000 - 2.3152 = 17.6848 \text{ ns}, \tag{8.3}$$

$$F_{\text{max,estimate}} \approx \frac{1}{17.6848 \text{ ns}} = 56.55 \text{ MHz}. \tag{8.4}$$

The frequency above is a constraint-equivalent estimate derived from the reported slack

rather than a separate sign-off Fmax analysis. The result proves that the mapped design satisfies the requested 50 MHz setup constraint in this synthesis model. A complete ASIC sign-off would additionally use extracted interconnect, propagated clocks, multi-corner setup and hold analysis, and a matching fast-corner HVT library [25, 26].

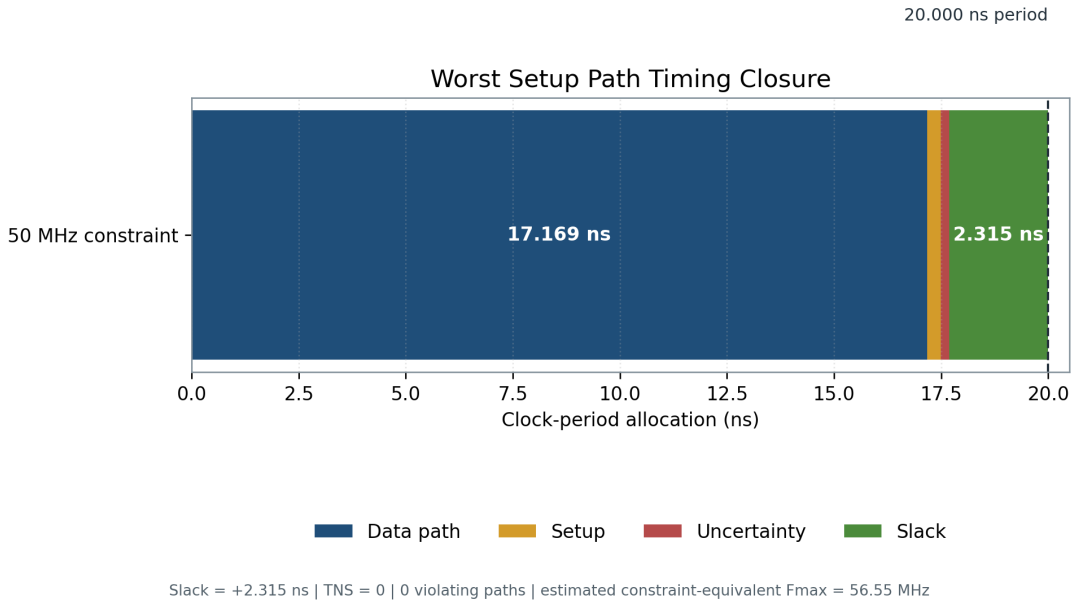


Figure 8.8: Worst setup-path allocation for the complete HVT-mapped SoC. The 20 ns clock period includes 17.169 ns of data-path delay, 0.316 ns of endpoint setup requirement, 0.200 ns of uncertainty, and 2.315 ns of remaining positive slack.

Table 8.7: Complete-SoC Cadence Genus HVT timing summary

Timing quantity	Value	Status
Clock period	20.000 ns	50 MHz target
Worst data path	17.169 ns	Register-to-register path
Setup requirement	0.316 ns	Included in required time
Clock uncertainty	0.200 ns	Applied by SDC
Worst setup slack	+2.315 ns	MET
Total negative slack	0.000 ns	No setup violations
Violating paths	0	Timing constraint satisfied
Estimated constraint-equivalent Fmax	56.55 MHz	Derived, not sign-off Fmax

8.3.3 Vectorless Power Estimate

Genus reports 1.43753 mW total power for the mapped SoC: 1.29801 mW internal power, 0.13743 mW switching power, and 0.002084 mW leakage power. Internal power contributes 90.29%, switching contributes 9.56%, and leakage contributes 0.14%. By cell category,

registers account for 1.15390 mW or 80.27% of the total, while combinational logic accounts for 0.283628 mW or 19.73% [5, 19].

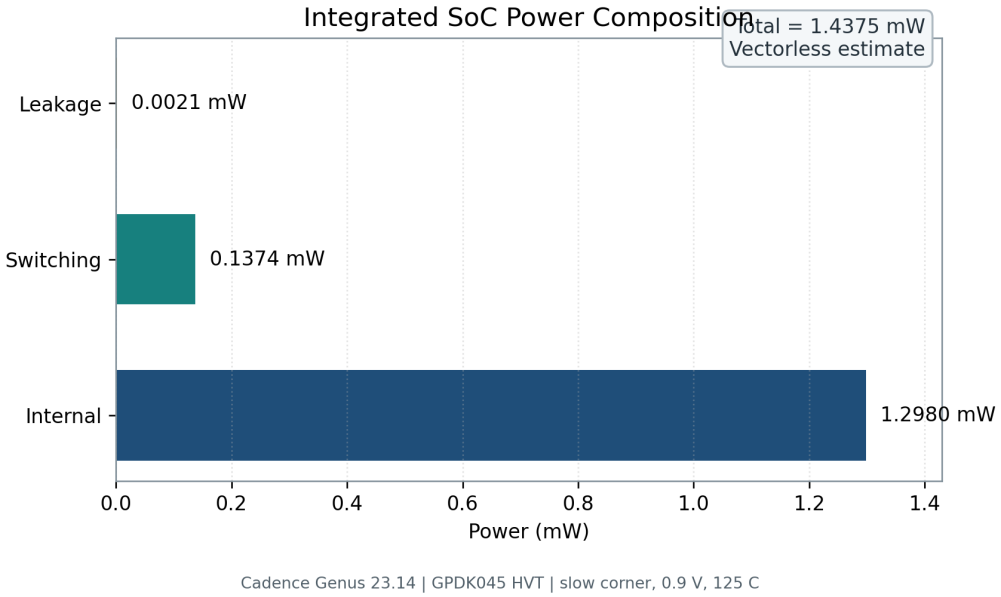


Figure 8.9: Cadence Genus vectorless power composition for the complete HVT-mapped SoC. The reported total is 1.43753 mW. This is a synthesis-stage estimate without application-derived VCD or SAIF activity and must not be compared directly with measured board power or the Quartus thermal-power estimate.

Table 8.8: Complete-SoC vectorless power reported by Cadence Genus

Component	Power	Share	Interpretation
Internal	1.29801 mW	90.29%	Cell-internal charging and short-circuit activity
Switching	0.13743 mW	9.56%	Estimated net-transition activity
Leakage	0.002084 mW	0.14%	Static HVT-cell leakage
Total	**1.43753 mW**	**100%**	Vectorless synthesis estimate

The very small leakage component is consistent with the intended HVT mapping, but it is not enough by itself to prove system energy efficiency. The run is vectorless, the clock network is not physically implemented, memories are synthesized from RTL rather than replaced by characterized SRAM macros, and interconnect parasitics are absent. A stronger next step is activity-based power using the full-SoC scenario, followed by Innovus placement and routing and Tempus multi-corner analysis. The present result should therefore be labelled as a low-leakage synthesis estimate, not post-layout silicon power [15, 19].

8.3.4 Mapped Schematic View

Figure 8.10 is the schematic exported from the same final Genus HVT session that produced the area, timing, power, netlist, SDC, and SDF reports. The view represents the complete mapped `riscv_aes_advancements` design after technology mapping. Its broad horizontal signal structure reflects the processor datapath and wide debug buses, while the dense central and right-hand regions reflect register banks, arithmetic and selection logic, AES transformation logic, data storage, MMIO routing, and peripheral control. The image is intentionally retained as tool-generated evidence; at this full-design scale it demonstrates connectivity and synthesis completeness rather than permitting individual gate inspection [1, 7].

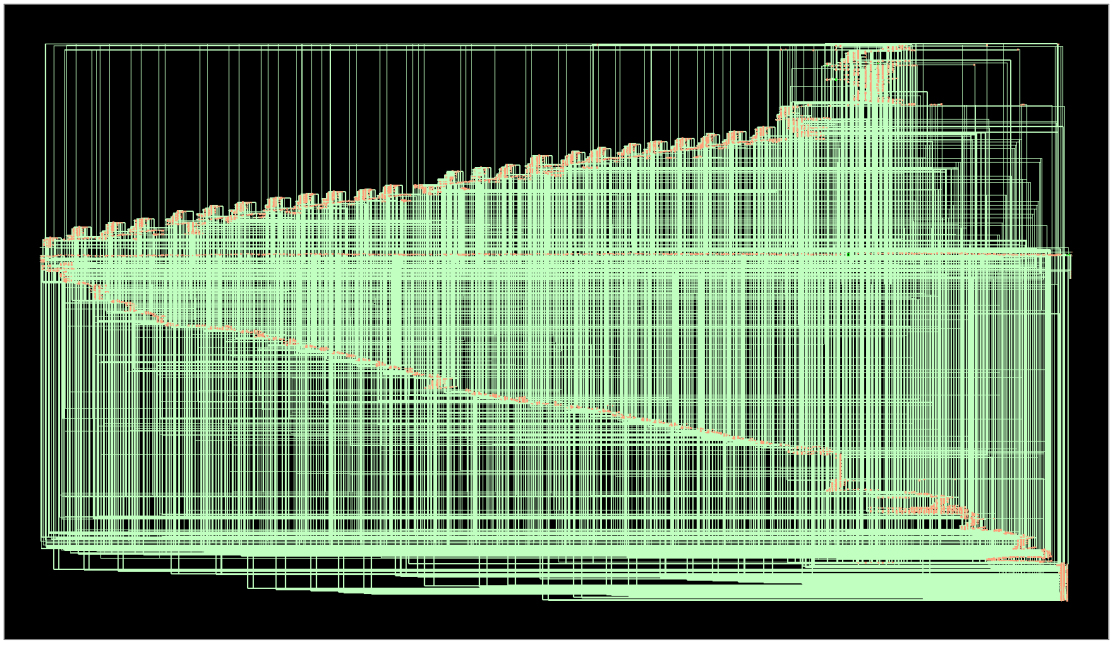


Figure 8.10: Cadence Genus GUI schematic of the complete HVT-mapped RISC-V AES security processor. The tool-generated view contains the technology-mapped processor pipeline, iterative AES subsystem, memory and MMIO network, sensor/SPI, UART, DMA, interrupt, power-management, and top-level debug paths. The dense appearance is consistent with the reported 50,968-leaf-cell implementation.

8.3.5 Generated Evidence and Reproducibility

The run completed with no unresolved references and no empty modules. Genus produced the mapped SystemVerilog netlist, generated SDC, SDF delay file, formal mapping database, and separate area, timing, power, QoR, and design-check reports. Runtime was approximately 1,088 s elapsed, with 2,865.68 MB peak memory on the Rocky Linux host. The antenna-cell warning concerns a physical-only cell that Genus marks unusable for logic mapping; the RTL attribute and datapath-optimization warnings do not invalidate elaboration or the final mapped reports, but they remain documented in the synthesis log [1, 7].

Table 8.9: Traceable outputs from the final complete-SoC HVT synthesis run

Evidence file	Purpose
`reports/report_check_design.rpt`	Confirms no unresolved references or empty modules
`reports/report_area.rpt`	Hierarchical mapped cell count and cell area
`reports/report_timing.rpt`	Full worst setup path and timing-point sequence
`reports/report_power.rpt`	Vectorless leakage, internal, switching, and category power
`reports/report_qor.rpt`	Consolidated timing, cell composition, area, runtime, and memory
`outputs/riscv_aes_advancements_netlist.sv`	Technology-mapped HVT gate-level netlist
`outputs/riscv_aes_advancements_synth.sdc`	Constraints emitted for downstream implementation
`outputs/delays.sdf`	Generated synthesis-stage delay annotation
`fv/riscv_aes_advancements/`	Formal RTL-to-mapped correspondence database
`gui_schematic_png.png`	Genus GUI export of the complete technology-mapped schematic

CHAPTER 9:

AUTONOMOUS VEHICLE APPLICATION

The selected application is a secure autonomous vehicle sensor gateway. A compact FPGA or ASIC block near the sensor interface reads safety-relevant telemetry, encrypts selected records with AES-CTR, transmits ciphertext to a vehicle gateway, and records interrupt and activity information. This use case is stronger than a simple wearable example because autonomous platforms contain many distributed sensing nodes and because unprotected telemetry can affect diagnostics, logging, and trust between vehicle subsystems [2, 4].

A practical example is an IMU and wheel-speed monitoring node. The plaintext record may contain a timestamp, acceleration, angular velocity, wheel speed, and diagnostic flags packed into a 128-bit record. The processor reads the sensor sample through SPI/MMIO, DMA-lite moves the record into the AES input path, AES-CTR encrypts it using a vehicle-session key and counter, and UART or another gateway-facing serial path transmits only ciphertext. The receiving gateway uses the same key, nonce, and counter to recover the plaintext and check whether the decoded data is consistent with expected vehicle behavior [4, 30].

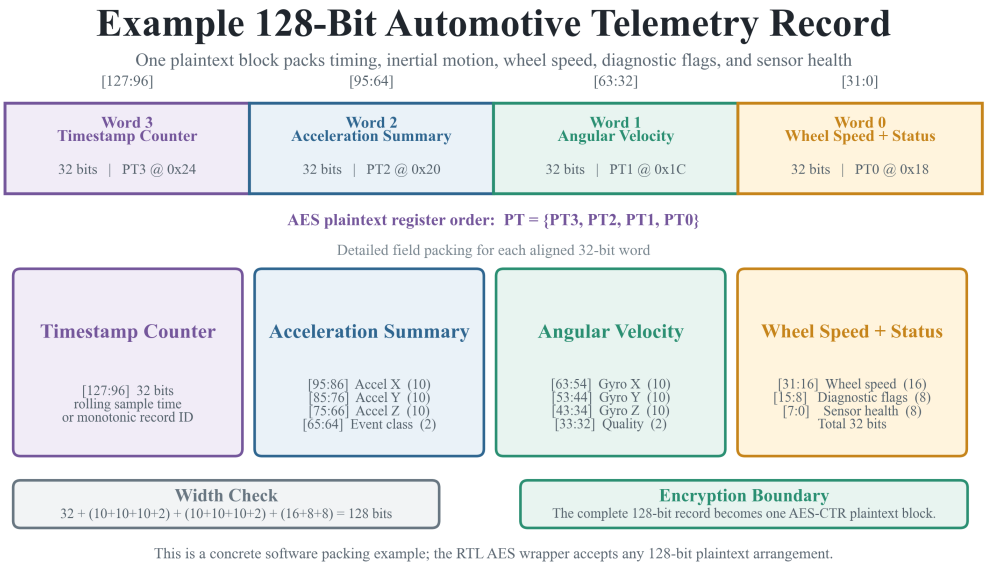


Figure 9.1: Example 128-bit automotive telemetry record mapped directly onto AES plaintext registers PT3 through PT0. PT3 carries a 32-bit sample timestamp or monotonic record identifier. PT2 packs three 10-bit acceleration values and a 2-bit event class. PT1 packs three 10-bit angular-velocity values and a 2-bit quality field. PT0 contains 16-bit wheel speed, 8-bit diagnostic flags, and 8-bit sensor-health status. The complete $32+32+32+32$-bit record forms one AES-CTR plaintext block.

The receiver decrypts by regenerating the same AES-CTR keystream. If the transmitted ciphertext block is C_i, the gateway computes $P_i = C_i \oplus E_K(N \parallel CTR_i)$. In the UVM environment this exact idea is checked by the independent C-DPI reference model, which compares RTL ciphertext and verifies that decrypted data matches the original randomized plaintext [2, 30].

9.1 Security Scope and Current Limitations

The present RTL is a transparent research prototype: it implements AES-CTR confidentiality, exposes nonce, counter, key, and interrupt state for verification, and does not claim message authentication, replay resistance, protected key storage, or complete machine-mode trap handling. These boundaries are important because functional AES correctness alone does not address active packet modification or physical leakage [11, 15].

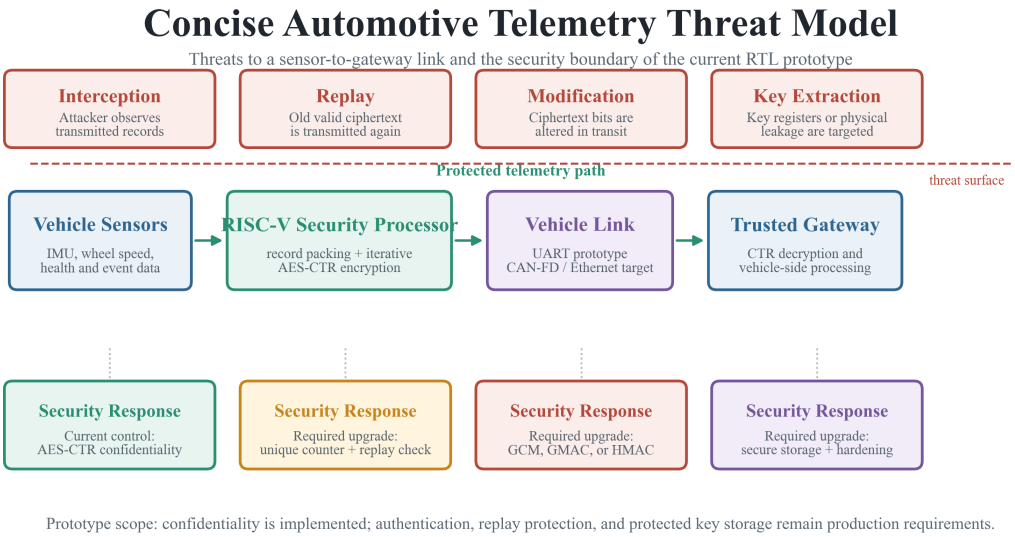


Figure 9.2: Automotive telemetry threat model and required responses. AES-CTR prevents a passive observer from directly reading intercepted records. Replay remains possible unless nonce-counter values are unique, persisted where necessary, and checked by the receiver. CTR ciphertext is malleable, so GCM, GMAC, or HMAC is needed to detect modification. Register-based keys remain observable and physically vulnerable; production hardware requires protected provisioning and storage, debug restrictions, and side-channel hardening.

9.2 Automotive Deployment Qualification

The demonstrated platform is an RTL/FPGA proof of concept rather than a qualified automotive electronic control unit. UART is retained as a compact, synthesizable laboratory transport; production deployment additionally requires authenticated protocol frames, protected provisioning, replay and fault handling, secure boot, diagnostic coverage, environmental validation, and lifecycle evidence aligned with automotive functional-safety and cybersecurity engineering processes such as ISO 26262 and ISO/SAE 21434 [10, 17].

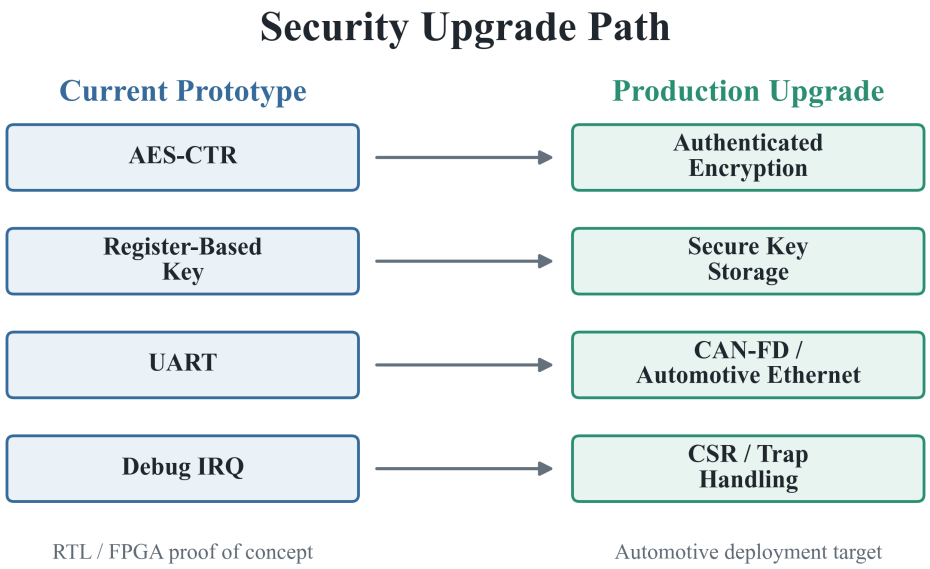


Figure 9.3: Prototype-to-production security upgrade path. The present AES-CTR path should become authenticated encryption; memory-mapped key registers should be replaced by protected key provisioning and storage; the demonstration UART should be replaced by CAN-FD or Automotive Ethernet; and the observable interrupt line should become complete privileged CSR, trap-vector, context-save, and interrupt-return handling.

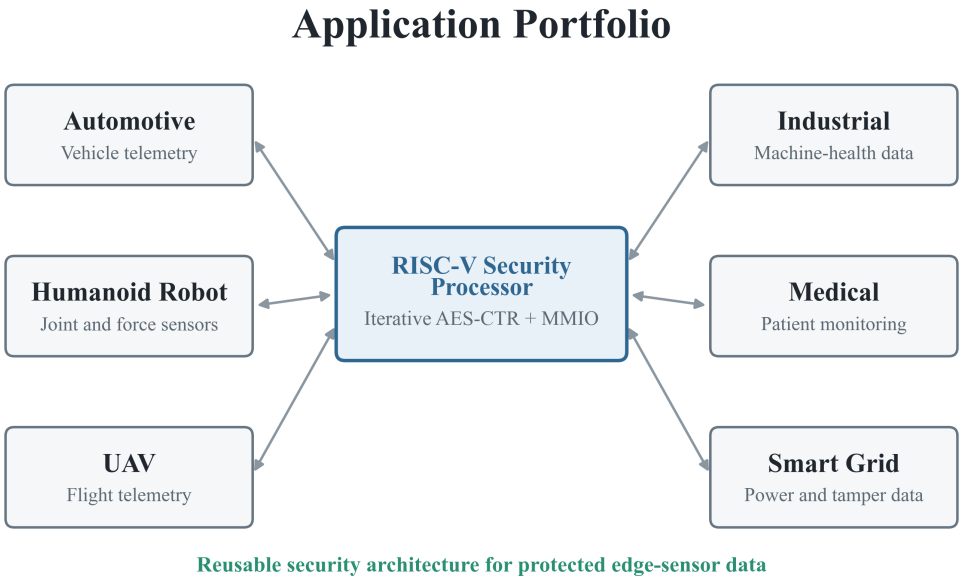


Figure 9.4: Application portfolio for the reusable security processor. The primary case is autonomous-vehicle telemetry containing IMU, wheel-speed, radar-metadata, and health records. The same moderate-rate edge-security architecture can protect humanoid joint and force feedback, UAV flight telemetry, industrial machine-health samples, privacy-sensitive medical-monitor records, and smart-grid voltage, current, tamper, and breaker-status data while leaving the CPU available for control tasks.

The main disadvantage is latency compared with a fully unrolled AES core. In a high-throughput camera pixel stream, this iterative core would not be the best encryption engine. The mitigation is to apply it to control telemetry, health data, metadata, event records, and low-to-moderate-rate sensor packets where the UART, gateway protocol, or sensor sampling rate is slower than the AES core. If higher throughput is later required, the design can be extended with multiple reusable AES lanes or a deeper pipelined AES while retaining the same RISC-V control plane [24, 27].

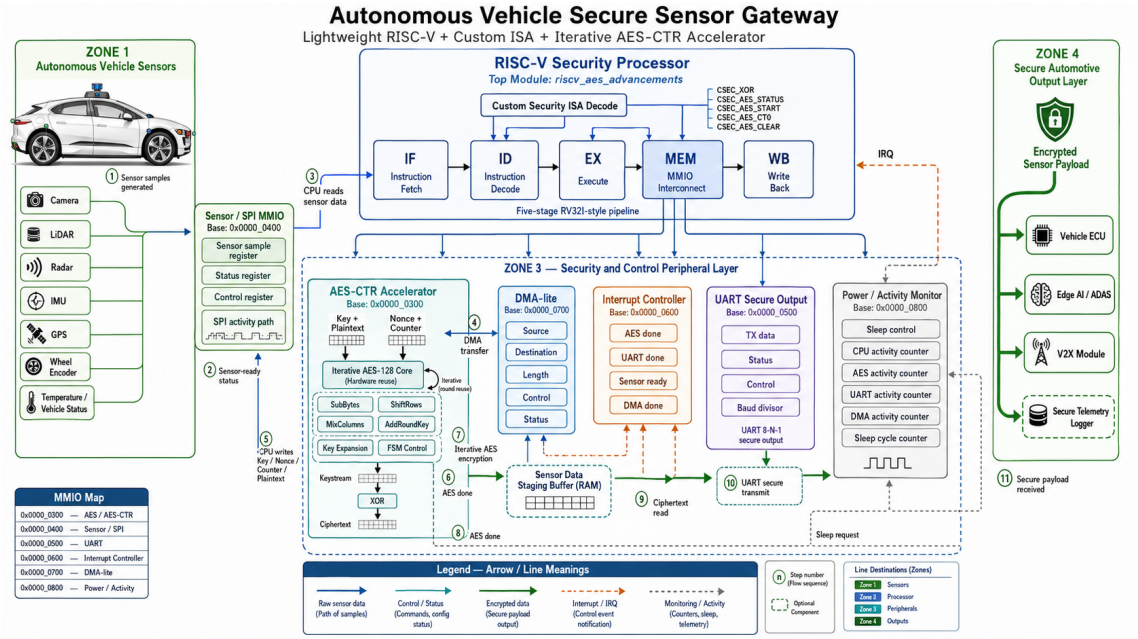


Figure 9.5: Autonomous vehicle secure sensor gateway application flow from sensor acquisition through CPU control, DMA staging, iterative AES-CTR encryption, UART transmission, interrupt monitoring, and secure payload delivery.

CHAPTER 10:

CONCLUSION AND FUTURE SCOPE

This work converts an iterative hardware-reusable AES-128 architecture into a complete RISC-V based IoT security processor. The standalone 55 nm AES results show strong area and power reduction compared with a baseline implementation. The integrated Questa and Quartus results show that the processor, AES-CTR, UART, sensor/SPI, DMA-lite, interrupt controller, activity counters, and custom ISA can be compiled, verified, and fitted together. The final GPDK045 HVT Genus run adds a distinct ASIC-style view of the complete SoC, reporting 50,968 mapped cells, 135,947.599 μm^2 cell area, +2.315 ns setup slack at 50 MHz, and 1.43753 mW vectorless total power. The project therefore links a publishable AES microarchitecture with a thesis-ready secure processor system while keeping each technology result in its proper context [3, 5].

The verification evidence is broad. Directed tests prove instruction and peripheral behavior, UVM proves randomized end-to-end encryption against an independent C model, portable functional coverage reaches 114 out of 114 bins over 100 transactions, and the scenario test proves the sensor-to-DMA-to-AES-to-UART-to-interrupt-to-sleep flow. The strongest result is not a single waveform but the consistency of these independent checks [1, 2].

Future extensions include authenticated encryption using GCM, AES-256 support, SHA or HMAC integration, secure boot, key storage with physical unclonable function support, side-channel countermeasures, intrusion-detection logic around abnormal sensor or bus activity, and a more complete RISC-V interrupt/CSR trap mechanism. For higher-end applications, the same control architecture can be scaled to automotive gateways, humanoid robots, industrial controllers, and UAV telemetry systems [15, 20].

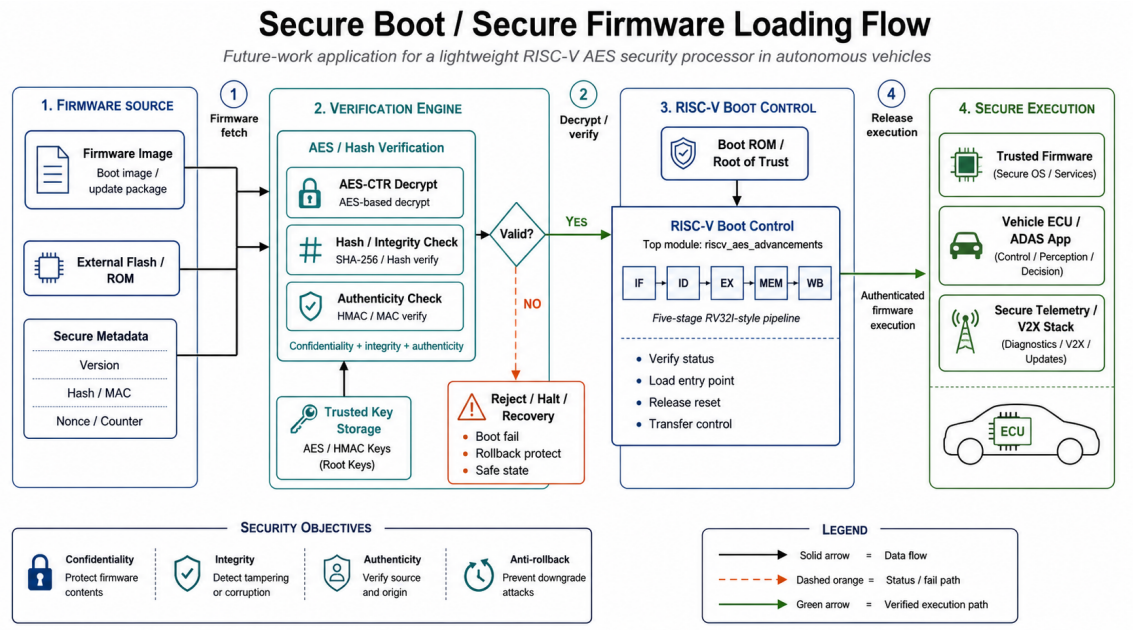


Figure 10.1: Future secure-boot and secure-firmware-loading extension. The proposed path combines AES-CTR decryption with hash or MAC verification, trusted key storage, boot control, rollback protection, and authenticated firmware release.

CHAPTER A:

COMPLETE MMIO REGISTER MAP

All peripheral addresses are byte addresses decoded by `mem_stage.sv`. Normal data-memory accesses remain on the RAM path, while the address ranges below select one peripheral and return its read value through the MEM-stage multiplexer.

Table A.1: Complete memory-mapped register map

Address	Register	Function
AES and AES-CTR, base 0x0000_0300		
0x0000_0300	`AES_CTRL`	Bit 0 starts encryption, bit 1 clears done, bit 2 selects CTR mode.
0x0000_0304	`AES_STATUS`	Bit 0 busy, bit 1 done, bit 2 CTR mode.
0x0000_0308–0314	`AES_KEY0--KEY3`	Four 32-bit words containing the 128-bit AES key.
0x0000_0318–0324	`AES_PT0--PT3`	Four 32-bit plaintext words.
0x0000_0328–0334	`AES_CT0--CT3`	Four 32-bit ciphertext result words.
0x0000_0338–033C	`AES_NONCE0--1`	64-bit CTR nonce.
0x0000_0340–0344	`AES_COUNT0--1`	64-bit CTR counter, incremented after each completed block.
Sensor and SPI, base 0x0000_0400		
0x0000_0400	`SENSOR_DATA`	Current sensor sample; writable for controlled test injection.
0x0000_0404	`SENSOR_STATUS`	Bit 0 indicates data ready.
0x0000_0408	`SENSOR_CONTROL`	Bit 0 enables the sensor interface and bit 1 clears data ready.
0x0000_0410	`SPI_RXDATA`	Most recently received SPI data.
0x0000_0414	`SPI_TXDATA`	Data written for SPI transmission.
0x0000_0418	`SPI_STATUS`	SPI status information.
0x0000_041C	`SPI_CONTROL`	SPI transfer control.
0x0000_0424	`SPI_SLAVE_SELECT`	Selects the active SPI sensor slave.
UART, base 0x0000_0500		
0x0000_0500	`UART_TXDATA`	Low byte starts an 8-N-1 transfer when the transmitter is idle.
0x0000_0504	`UART_STATUS`	Bit 0 busy and bit 1 done.
0x0000_0508	`UART_CONTROL`	Bit 0 enable and bit 1 clear done.

Address	Register	Function
0x0000_050C	UART_BAUD_DIV	UART terminal count; each serial bit lasts BAUD_DIV+1 enabled system-clock cycles.
Interrupt controller, base 0x0000_0600		
0x0000_0600	IRQ_PENDING	Bits 0–3 record AES, UART, sensor, and DMA events.
0x0000_0604	IRQ_ENABLE	Enables the corresponding pending sources.
0x0000_0608	IRQ_CLEAR	Writing one clears the corresponding pending bit.
DMA-lite, base 0x0000_0700		
0x0000_0700	DMA_SRC_ADDR	Source byte address.
0x0000_0704	DMA_DST_ADDR	Destination byte address.
0x0000_0708	DMA_LEN	Transfer length in 32-bit words.
0x0000_070C	DMA_CTRL	Bit 0 start and bit 1 clear done.
0x0000_0710	DMA_STATUS	Bit 0 busy and bit 1 done.
Power and activity, base 0x0000_0800		
0x0000_0800	POWER_CTRL	Bit 0 requests sleep and bit 1 clears activity counters.
0x0000_0804	CPU_ACTIVE_CYCLES	Counts CPU-active clock cycles.
0x0000_0808	AES_ACTIVE_CYCLES	Counts AES-active clock cycles.
0x0000_080C	UART_ACTIVE_CYCLES	Counts UART-active clock cycles.
0x0000_0810	SLEEP_CYCLES	Counts cycles while sleep is requested.
0x0000_0814	DMA_ACTIVE_CYCLES	Counts DMA-active clock cycles.
0x0000_0818	SENSOR_ACTIVE_CYCLES	Counts sensor-interface active cycles.

CHAPTER B:

CUSTOM SECURITY ISA ENCODING

The custom security instructions use the RISC-V `custom-0` opcode. They supplement rather than replace the standard MMIO path, allowing the thesis to compare software-visible register transactions with instruction-triggered accelerator control.

Table B.1: Custom-0 instruction field allocation

Bits	Field	Meaning
31:25	funct7	Reserved for future security-operation expansion
24:20	rs2	Second source operand
19:15	rs1	First source operand
14:12	funct3	Security command selector
11:7	rd	Destination register
6:0	opcode	`0001011`, RISC-V custom-0

Table B.2: Implemented custom security commands

funct3	Mnemonic	Behavior
000	CSEC_XOR	Writes `rs1XORrs2` to `rd`.
001	CSEC_AES_STATUS	Returns AES busy, done, and selected mode.
010	CSEC_AES_START	Starts AES-CTR using low plaintext words from `rs1` and `rs2`.
011	CSEC_AES_CT0	Returns ciphertext bits 31:0.
100	CSEC_AES_CLEAR	Clears the AES completion state.

CHAPTER C:

BUILD AND VERIFICATION COMMANDS

Table C.1: Reproducible project commands

Purpose	**Command and expected evidence**
Complete integrated RTL regression	`powershell -ExecutionPolicy Bypass -File ./run_questa_regression.ps1` *Evidence:* 107 checks pass with zero failures.
AES known-answer capture	`./verification/questa_wave_capture/launch_capture.ps1 -Test 01` *Evidence:* Native Questa waveform and AES transcript.
Directed CPU capture	`./verification/questa_wave_capture/launch_capture.ps1 -Test 02` *Evidence:* Instruction and pipeline waveforms.
Peripheral capture	`./verification/questa_wave_capture/launch_capture.ps1 -Test 03` *Evidence:* AES-CTR, SPI, UART, IRQ, DMA, power, and custom-ISA activity.
Random smoke capture	`./verification/questa_wave_capture/launch_capture.ps1 -Test 04` *Evidence:* Changing randomized values and addresses.
UVM end-to-end run	`./verification/uvm_e2e/run_uvm_e2e_questa.ps1 -NumTxns 25 -Seed 101` *Evidence:* C-DPI comparison and UART scoreboard results.
Coverage closure	`run_uvm_e2e_questa.ps1 -NumTxns 100 -Seed 611` *Evidence:* 114/114 portable bins and 100 UART matches.
Full-SoC scenario	`./verification/scenarios/run_secure_health_monitoring_scenario.ps1` *Evidence:* 15 scenario checks pass.
Quartus full FPGA flow	`powershell -File ./compile_full_low_power.ps1` *Evidence:* Map, fit, assembler, and timing reports.
Quartus power analysis	`quartus_pow riscv_aes_advancements -c riscv_aes_advancements` *Evidence:* Post-fit `pow.rpt` and `pow.summary`.
Standalone AES Genus flow	`genus -f genus_script.tcl` *Evidence:* 55 nm baseline-versus-reusable AES comparison reports.

Purpose	Command and expected evidence

Complete-SoC HVT Genus flow	`export GENUS_LIB=/path/to/slow_vdd1v0_basicCells_hvt.lib`
	`genus -f genus_script.tcl`
	Evidence: HVT area, timing, power, QoR, mapped netlist, SDC, SDF, and FV database.

CHAPTER D:

UVM CLASS HIERARCHY AND DATA FLOW

Table D.1: Classes in the end-to-end UVM environment

Class	UVM role	Responsibility
`uvm\_e2e\_item`	Sequence item	Carries randomized plaintext, key, nonce, counter, and comparison results.
`uvm\_uart\_line`	Sequence item	Carries expected or reconstructed UART text.
`uvm\_e2e\_sequence`	Sequence	Generates deterministic pseudo-random transactions from a reproducible seed.
`uvm\_e2e\_sequencer`	Sequencer	Supplies transactions to the driver.
`uvm\_e2e\_driver`	Driver	Calls the C-DPI model, programs AES MMIO, reads ciphertext, and transmits the result through UART.
`uvm\_uart\_monitor`	Monitor	Samples the physical UART TX waveform and reconstructs newline-terminated text.
`uvm\_e2e\_scoreboard`	Scoreboard	Compares observed UART lines with expected lines and records matches or mismatches.
`uvm\_e2e\_coverage`	Subscriber	Measures input bins, cross bins, cryptographic matches, and portable coverage closure.
`uvm\_e2e\_env`	Environment	Instantiates and connects sequencer, driver, monitor, scoreboard, and coverage.
`uvm\_e2e\_test`	Test	Starts the sequence and controls the UVM run-phase objection.

Listing D.1: End-to-end UVM information flow

```
randomized sequence item
    -> UVM driver
       -> independent C-DPI AES-CTR reference
       -> RTL AES MMIO and reusable AES core
       -> RTL UART transmitter
    -> UART monitor reconstructs serial bytes
    -> scoreboard compares expected and observed lines
    -> coverage collector records values, crosses, and match bins
```

CHAPTER E:

RESULT-TO-SOURCE TRACEABILITY

Table E.1: Traceability of major thesis claims

Claim	**Reported result**	**Primary evidence**
AES functional correctness	NIST AES-128 ciphertext matches	`verification/questa\_wave\_capture/results/01\_aes\_known\_answer\_capture.log`
Integrated RTL regression	107 passes, zero failures	`output\_files/transcript.txt`
UVM end-to-end comparison	RTL equals C-DPI reference and UART line matches	`verification/results/2026-06-15\_reusable\_aes/05\_uvm\_e2e\_25\_seed101.log`
Functional coverage	114/114 bins and 100 UART matches	`verification/results/2026-06-15\_reusable\_aes/06\_functional\_coverage\_100\_seed611.log`
Full-SoC scenario	15 checks pass	`verification/results/2026-06-15\_reusable\_aes/07\_full\_soc\_scenario.log`
Standalone proposed AES area	2,694 cells and 11,841.480 μm^2	`GCONpapercode/result/reports/report\_area.rpt`
Standalone proposed AES power	8.53125×10^{-4} W	`GCONpapercode/result/reports/report\_power.rpt`
Integrated FPGA resources	13,257 ALMs and 11,154 registers	`output\_files/riscv\_aes\_advancements.fit.summary`
Integrated FPGA timing	2.577 ns setup and 0.371 ns hold slack	`output\_files/riscv\_aes\_advancements.sta.summary`
Integrated FPGA power estimate	519.72 mW total thermal power	`output\_files/riscv\_aes\_advancements.pow.summary`
Complete-SoC HVT synthesis	50,968 cells and 135,947.599 μm^2	`risc_aes_finalReports/reports/report_area.rpt`
Complete-SoC HVT timing	+2.315 ns setup slack at 50 MHz, TNS 0	`risc_aes_finalReports/reports/report_timing.rpt`
Complete-SoC HVT vectorless power	1.43753 mW total, 0.002084 mW leakage	`risc_aes_finalReports/reports/report_power.rpt`
Complete-SoC HVT deliverables	Mapped netlist, synthesized SDC, SDF and formal map	`risc_aes_finalReports/outputs/` and `fv/`

REFERENCES

[1] T. S. Gampala, A. C. Pamarthy, and C. S. Reddy, “Designing of AES-128 encryption algorithm using System Verilog VLSI,” in Proc. IEEE Wireless Antenna and Microwave Symposium (WAMS), 2025.

[2] S. Kannan, K. Koundinya, and B. K. Panigrahi, “Machine learning-based detection of attacks on AES cryptography using FPGA power traces,” in Proc. Int. Conf. Intelligent Computing and Control Systems (ICICCS), 2025.

[3] P. Priya, P. Karthigaikumar, and S. S. Sridevi, “Hardware efficient realization of 128-bit Advanced Encryption Standard in FPGA,” in Proc. Int. Conf. Intelligent Systems and Machine Learning (ISML), 2024.

[4] D. M. Hamand et al., “FPGA based hardware accelerator for secure IoT edge computing using AES,” in Proc. IEEE Conf. on IoT and Edge Computing, 2024.

[5] K. B. Sarmila, A. R. Kaarthikeyan, and A. Kaasiprasanth, “Optimized AES for low power devices,” in Proc. 10th Int. Conf. Advanced Computing and Communication Systems (ICACCS), 2024.

[6] P. Prakashan et al., “A configurable AES implementation on FPGA for secure 5G systems,” in Proc. IEEE Conf. on 5G Systems, 2023.

[7] T. A. S. et al., “Design of Advanced Encryption Standard using Verilog HDL,” in Proc. Int. Conf. on Electronics and Informatics (ICOEI), 2023.

[8] Y. Xu et al., “Unified coprocessor for high-speed AES-128 and SM4 encryption,” in Proc. Int. Conf. on Integrated Circuits, Technologies and Applications (ICTA), 2022.

[9] N. R. et al., “Lightweight implementation of the AES encryption algorithm for IoT applications constrained by memory and processing power,” in Proc. IEEE Conf. on Microelectronics and IoT, 2022.

[10] International Organization for Standardization and SAE International, “ISO/SAE 21434:2021 Road vehicles—Cybersecurity engineering,” Geneva, Switzerland, 2021.

[11] M. N. A. et al., “Software implementation of AES-128 side-channel attacks based on power traces decomposition,” in Proc. IEEE Conf. on Information Security and Forensics, 2021.

[12] S. Potluri et al., “Dynamically reconfigurable resource efficient AES implementation for IoT applications,” in Proc. IEEE Int. Symp. on Circuits and Systems (ISCAS), 2020.

[13] S. R. et al., “Dynamically reconfigurable resource efficient AES implementation for IoT applications,” in Proc. IEEE ISCAS, 2020.

[14] S. S. K. Das, P. K. Dutta, and S. Chattopadhyay, “A ResNet-based DVFS regulator for heterogeneous multi-core mobile processors,” in Proc. IEEE Int. Symp. on Low Power Electronics and Design (ISLPED), 2020.

[15] Y.-H. Chou and S.-L. L. Lu, “A high performance, low energy, compact masked 128-bit AES in 22 nm CMOS technology,” in Proc. IEEE Int. Symp. on Circuits and Systems (ISCAS), 2019.

[16] M. K. Bange et al., “Dynamically reconfigurable power efficient security for Internet of Things devices,” in Proc. MOCAST on Modern Circuits and Systems Technologies, 2018.

[17] International Organization for Standardization, “ISO 26262-1:2018 Road vehicles—Functional safety—Part 1: Vocabulary,” Geneva, Switzerland, 2018.

[18] V. Vo, “The merged clock gating architecture for low power digital clock application on FPGA,” in Proc. Int. Conf. on Advanced Technologies for Communications (ATC), 2018.

[19] S. Kerckhof et al., “Towards green cryptography: A comparison of lightweight ciphers from the energy viewpoint,” in Proc. IEEE Int. Symp. on Hardware-Oriented Security and Trust (HOST), 2012.

[20] A. Moradi et al., “Pushing the limits: A very compact and a threshold implementation of AES,” in Proc. IEEE Int. Conf. on Reconfigurable Computing and FPGAs (ReConFig), 2011.

[21] Texas Instruments, “KeyStone Architecture Universal Asynchronous Receiver/Transmitter (UART) User Guide,” Literature No. SPRUGP1, Nov. 2010.

[22] R. Chaves and G. Kuzmanov, “Low-cost hardware architecture for AES encryption in embedded systems,” IEEE Transactions on Computers, vol. 57, no. 9, pp. 1167–1178, 2008.

[23] S. Kaur and R. Vig, “Efficient implementation of AES algorithm in FPGA device,” in Proc. Int. Conf. on Computational Intelligence and Multimedia Applications (ICCIMA), 2007.

[24] M. McLoone and J. V. McCanny, “High-performance FPGA implementation of AES encryption and decryption,” in Proc. IEEE Int. Conf. on Field-Programmable Technology (FPT), 2007.

[25] S. Tillich and J. Grossschadl, “A survey of FPGA-based AES implementations,” in Proc. IEEE Design, Automation and Test in Europe Conference (DATE), 2006.

[26] T. Good and M. Benaissa, “AES on FPGA from the fastest to the smallest,” in Proc. IEEE Int. Conf. on Field-Programmable Technology (FPT), 2005.

[27] A. Hodjat and I. Verbauwhede, “Area-throughput trade-offs for fully pipelined AES processors,” in Proc. IEEE Int. Conf. on Field-Programmable Technology (FPT), 2004.

[28] K. Gaj and P. Chodowiec, “Comparison of the hardware performance of the AES candidates using reconfigurable hardware,” in Proc. IEEE Int. Conf. on Field-Programmable Technology (FPT), 2001.

[29] A. Satoh, S. Morioka, K. Takano, and S. Munetoh, “A compact Rijndael hardware architecture with S-box optimization,” IEEE Transactions on Circuits and Systems II: Express Briefs, vol. 48, no. 10, pp. 938–942, 2001.

[30] J. Daemen and V. Rijmen, “AES proposal: Rijndael,” in Proc. IEEE Int. Conf. on Computer Design (ICCD), 2000.